

Sockets

1 Introdução

Sockets são abstrações de programação que permitem a troca de dados (bytes) entre processos através de uma rede. Eles são o canal de comunicação subjacente. Para que o RPC funcione usando *sockets*, é necessário construir um protocolo acima deles (para definir como as chamadas de procedimento são estruturadas, enviadas e respondidas). Portanto, *sockets* são a infraestrutura de transporte, não o protocolo de chamada remota em si.

Sockets são mecanismos de transporte de baixo nível que podem ser usados para *implementar / construir* sistemas RPC ou RMI, mas não são RPC em si. A **porta** é apenas um identificador numérico (um número de 1 a 65535) dentro do seu próprio sistema operacional. Ela serve para o seu computador saber para qual programa enviar os dados que chegam da internet. Uma analogia, uma porta é a vaga de estacionamento no servidor. O *socket* é o carro do cliente encaixado nessa vaga, criando a conexão ativa. Na prática, fala-se porta 80 e *socket* 192.168.0.10:80.

2 Instalação dos Pré-Requisitos

Abra o GitBash, vá para a pasta onde deseja armazenar o projeto (supondo que a pasta `programas` abaixo já existe):

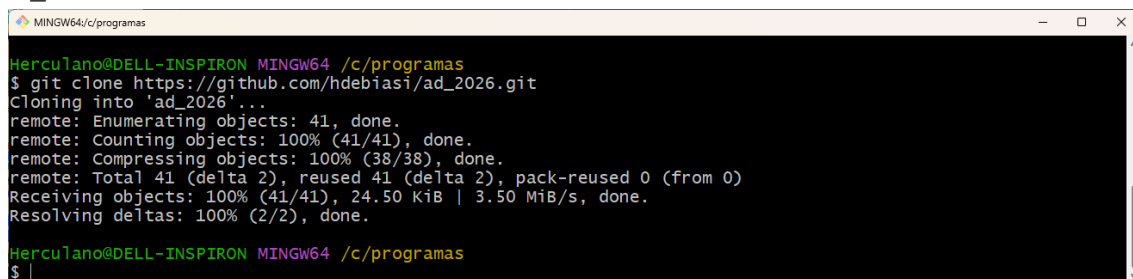
- Linux / macOS:
 - `cd ~/programas`
- Windows (Git Bash):
 - `cd /c/programas`
- Windows (cmd ou PowerShell):
 - `cd /d c:\programas`

Clone o repositório:

- `git clone https://github.com/hdebiasi/ad\_2026.git`

Entre na pasta do projeto:

- `cd ad_2026`

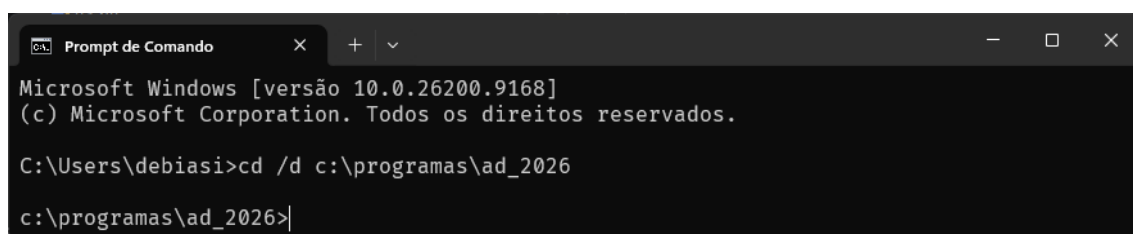


```
MINGW64/c/programas
Herculano@DELL-INSPIRON MINGW64 /c/programas
$ git clone https://github.com/hdebiasi/ad_2026.git
Cloning into 'ad_2026'...
remote: Enumerating objects: 41, done.
remote: Counting objects: 100% (41/41), done.
remote: Compressing objects: 100% (38/38), done.
remote: Total 41 (delta 2), reused 41 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (41/41), 24.50 KiB | 3.50 MiB/s, done.
Resolving deltas: 100% (2/2), done.
Herculano@DELL-INSPIRON MINGW64 /c/programas
$
```

A seguir siga uma das seções abaixo (2.1 ou 2.2) para configurar o ambiente virtual no cmd ou no PyCharm (preferível).

2.1 Ambiente Virtual (Linha de Comando)

Abra algum terminal e certifique-se de estar dentro da pasta clonada do projeto:



```
Prompt de Comando
Microsoft Windows [versão 10.0.26200.9168]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\debiasi>cd /d c:\programas\ad_2026

c:\programas\ad_2026>
```

Execute as instruções abaixo dependendo do sistema operacional e terminal usado:

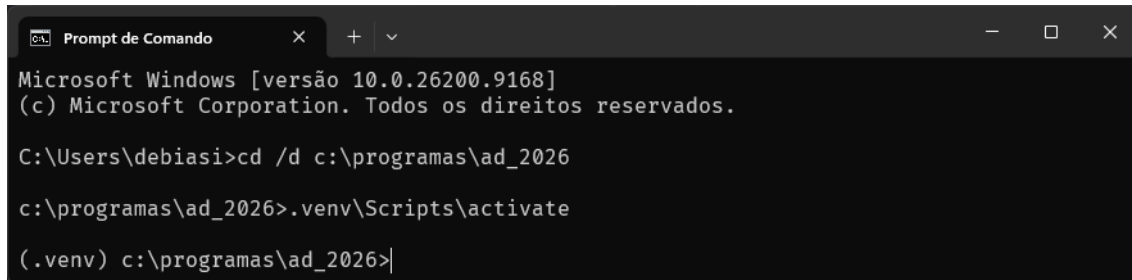
```
# 1. Criar o ambiente virtual nativo (substitui o 'conda create')
python3 -m venv .venv

# 2. Ativar o ambiente virtual (substitui o 'conda activate')
# No Linux/macOS:
source .venv/bin/activate

# No Windows (cmd ou PowerShell):
# .venv\Scripts\activate

# No Windows (Git Bash):
# source .venv/Scripts/activate
```

Neste ponto é importante certificar que o *prompt* mostra que o ambiente virtual (.venv) está ativado:

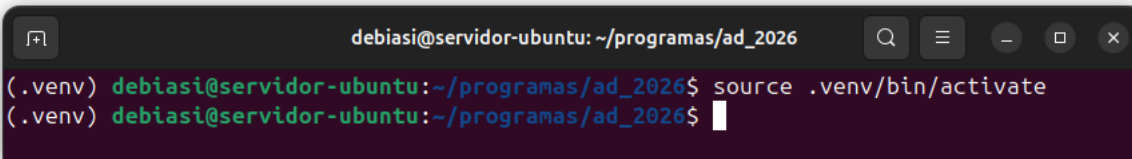


```
Microsoft Windows [versão 10.0.26200.9168]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\debiasi>cd /d c:\programas\ad_2026

c:\programas\ad_2026>.venv\Scripts\activate

(.venv) c:\programas\ad_2026>|
```



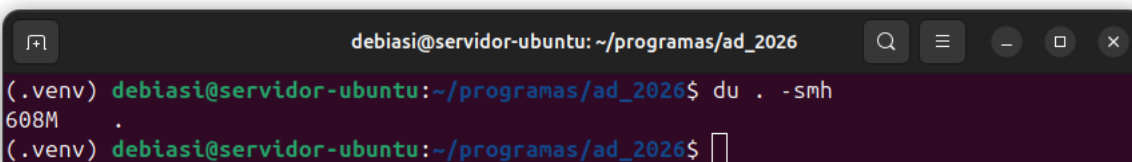
```
debiasi@servidor-ubuntu: ~/programas/ad_2026

(.venv) debiasi@servidor-ubuntu:~/programas/ad_2026$ source .venv/bin/activate
(.venv) debiasi@servidor-ubuntu:~/programas/ad_2026$ |
```

```
# 3. Atualizar o gerenciador de pacotes pip (Boa prática)
pip install --upgrade pip

# 4. Instalar todas as bibliotecas de uma vez via PIP
pip install streamlit streamlit-folium geopy geopandas folium python-dotenv requests httpx
```

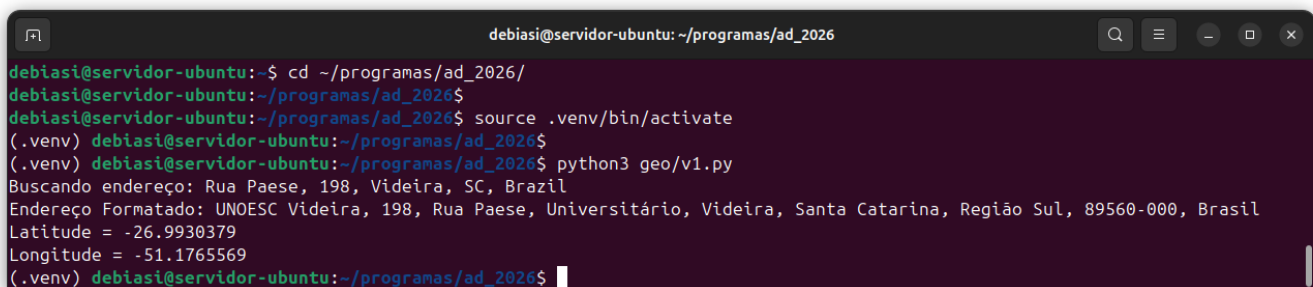
A criação do ambiente virtual irá ocupar bastante espaço em disco, entre 500 MB e 600 MB:



```
debiasi@servidor-ubuntu: ~/programas/ad_2026

(.venv) debiasi@servidor-ubuntu:~/programas/ad_2026$ du . -smh
608M  .
(.venv) debiasi@servidor-ubuntu:~/programas/ad_2026$ |
```

Na linha de comando, sempre ao fechar e reabrir o terminal, para executar os programas será necessário entrar dentro da pasta do projeto e ativar o ambiente virtual:



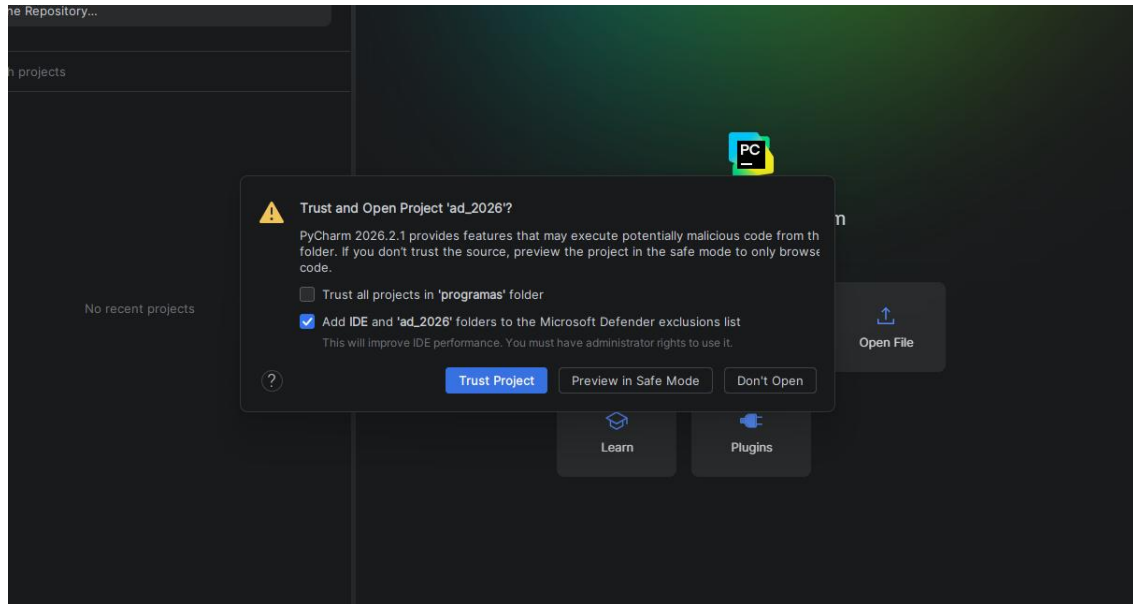
```
debiasi@servidor-ubuntu:~$ cd ~/programas/ad_2026/
debiasi@servidor-ubuntu:~/programas/ad_2026$
debiasi@servidor-ubuntu:~/programas/ad_2026$ source .venv/bin/activate
(.venv) debiasi@servidor-ubuntu:~/programas/ad_2026$
(.venv) debiasi@servidor-ubuntu:~/programas/ad_2026$ python3 geo/v1.py
Buscando endereço: Rua Paese, 198, Videira, SC, Brazil
Endereço Formatado: UNOESC Videira, 198, Rua Paese, Universitário, Videira, Santa Catarina, Região Sul, 89560-000, Brasil
Latitude = -26.9930379
Longitude = -51.1765569
(.venv) debiasi@servidor-ubuntu:~/programas/ad_2026$ |
```

Para desativar o ambiente virtual, execute:

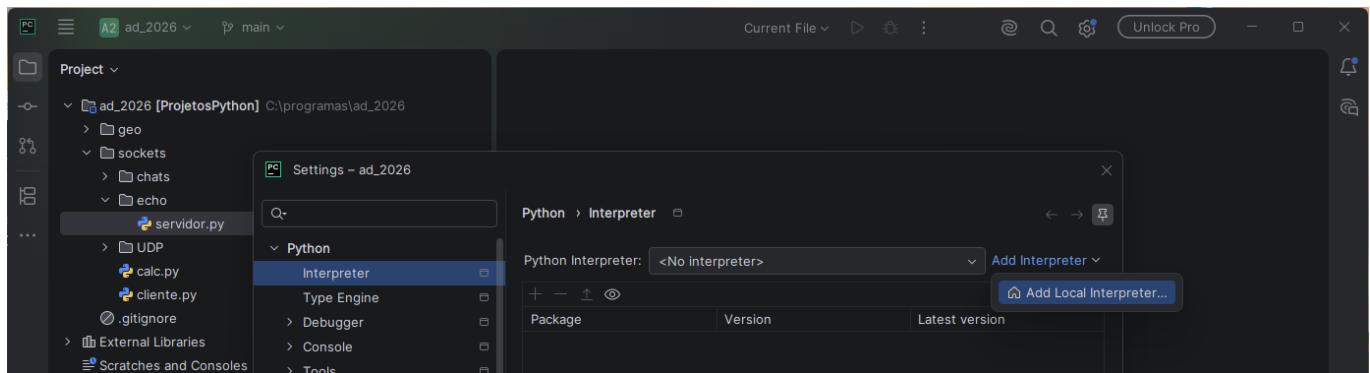
➤ deactivate

2.2 Configurando o Projeto no PyCharm

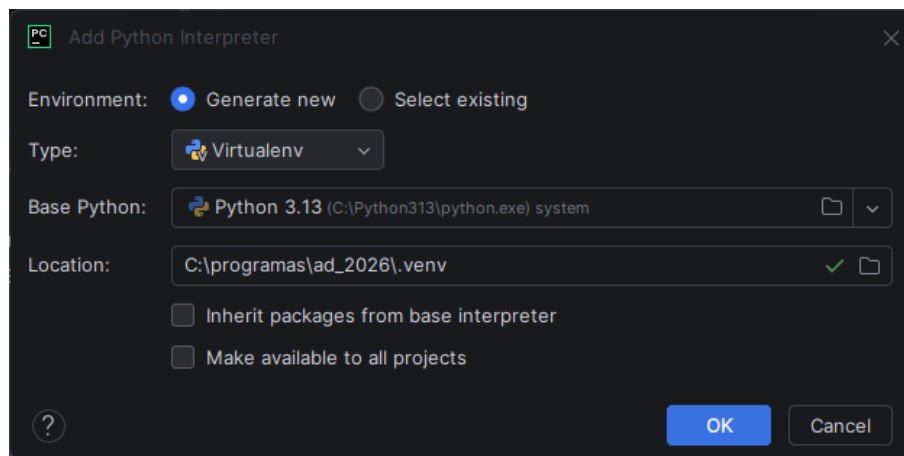
Abra o projeto no PyCharm:



Vá em *File* → *Settings* → *Python* → *Interpreter* e clique em *Add Local Interpreter ...*



Clique em *Ok* e novamente em *Ok*:



Abra o terminal embutido no PyCharm (*Alt + F12*) e instale as bibliotecas dentro do ambiente virtual:

- `pip install streamlit streamlit-folium geopy geopandas folium python-dotenv requests httpx`

3 Servidor e Cliente Eco (TCP)

Código do servidor:

```
import sys
import socket

# Verifica se o IP foi passado como argumento, senão usa localhost por padrão
HOST = sys.argv[1] if len(sys.argv) > 1 else '0.0.0.0'
PORT = 5000

# Cria o socket TCP
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Permite reusar a porta imediatamente após fechar o servidor
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

# Vincula o servidor ao IP local e à porta 5000
server.bind((HOST, PORT))

# Define o limite de conexões em espera
server.listen(1)
print(f"[-] Servidor rodando em {HOST}:{PORT}... Pressione Ctrl+C para interromper.")

try:
    # Bloqueia a execução até que um cliente se conecte e aceita a conexão do cliente
    conexao, cliente_ip = server.accept()
    print(f"[+] Conectado por: {cliente_ip}")

    # Loop para receber dados do cliente
    while True:
        # Recebe até 1024 bytes
        dados = conexao.recv(1024)
        if not dados:
            # Se o cliente der Ctrl+C no servidor, o socket recebe dados vazios e fecha
            print("[*] Conexão encerrada pelo cliente.")
            break

        # Mostra mensagem original
        print(f"[Recebido do cliente]: {repr(dados)}")

        # Decodifica os bytes da rede para string usando UTF-8
        mensagem_decodificada = dados.decode('utf-8')

        # .strip() remove o '\n' enviado pelo Enter do Netcat e espaços extras
        mensagem_limpa = mensagem_decodificada.strip()

        # Se a mensagem digitada (sem o \n) for 'sair', quebra o loop e fecha o servidor
        if mensagem_limpa.lower() == 'sair':
            print("[*] Comando 'sair' recebido. Finalizando servidor...")
            # Envia um aviso de despedida antes de fechar (opcional)
            conexao.sendall("Conexão encerrada pelo servidor.\n".encode('utf-8'))
            break

        # Converte para maiúsculas
        mensagem_maiuscula = mensagem_limpa.upper()

        print(f"[Mensagem limpa]: {mensagem_limpa} -> [Processada]: {mensagem_maiuscula}")

        # Codifica de volta para bytes em UTF-8 antes de enviar pelo socket
        # Devolve (eco) de volta o mesmo texto em maiúsculas (Eco Modificado)
        # Eco de volta com uma quebra de linha para o terminal do Netcat ficar organizado
        resposta = mensagem_maiuscula + "\n"
        conexao.sendall(resposta.encode('utf-8'))

    print("[-] Fechando conexão.")
    conexao.close()

# CASO 2: Ctrl+C pressionado no próprio SERVIDOR
except KeyboardInterrupt:
    print("\n[*] Ctrl+C detectado! Desligando o servidor com segurança...")

finally:
    # Garante que o socket principal do servidor seja fechado em qualquer situação
    server.close()
    print("[*] Porta liberada. Finalizado.")
```

Código do cliente:

```
import sys
import socket

# 1. Configuração dos valores padrões
HOST = '127.0.0.1'
PORT = 5001

# 2. Tratamento dos argumentos da linha de comando
if len(sys.argv) > 1:
    HOST = sys.argv[1] # Se fornecido, substitui o IP padrão

if len(sys.argv) > 2:
    try:
        PORT = int(sys.argv[2]) # Se fornecido, substitui a porta padrão
    except ValueError:
        print("Erro: A porta deve ser um número inteiro.")
        print("Uso correto: python3 cliente.py <IP_DO_SERVIDOR> <PORTA>")
        print("Exemplo:      python3 cliente.py 192.168.1.50 10000")
        sys.exit(1)

# 3. Criação do socket TCP
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

try:
    print(f"[*] Tentando conectar a {HOST}:{PORT}...")
    client.connect((HOST, PORT))
    print(f"[+] Conectado com sucesso!\n")
    print("Digite suas mensagens, contas ou 'sair' para encerrar.")
    print("-" * 50)

    # 4. Loop de envio e recepção
    while True:
        texto = input("Envio > ")

        # Remove espaços desnecessários antes de checar e enviar
        texto_limpo = texto.strip()

        if not texto_limpo:
            continue

        if texto_limpo.lower() == 'sair':
            # Envia o comando de saída para o servidor também saber que terminou
            client.sendall(texto_limpo.encode('utf-8'))
            break

        # Envia a string codificada em bytes para o socket
        client.sendall(texto_limpo.encode('utf-8'))

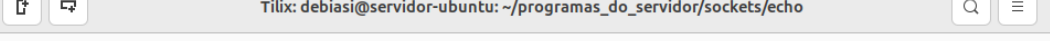
        # Aguarda a resposta do servidor (seja o Eco ou o resultado do cálculo)
        resposta_bytes = client.recv(1024)
        if not resposta_bytes:
            print("[*] O servidor encerrou a conexão.")
            break

        resposta_texto = resposta_bytes.decode('utf-8').strip()
        print(f"Retorno < {resposta_texto}\n")
    except KeyboardInterrupt:
        print("\n[*] Conexão interrompida pelo usuário (Ctrl+C).")
    except ConnectionRefusedError:
        print(f"[-] Erro: Conexão recusada em {HOST}:{PORT}.")
        print("Certifique-se de que o servidor correspondente está rodando nesta porta.")
    except Exception as e:
        print(f"[-] Ocorreu um erro inesperado: {e}")
    finally:
        client.close()
        print("[*] Socket fechado. Programa finalizado.")
```

Exercício: Use o código acima e a função `eval()` do Python para transformar o código acima em uma calculadora. Ela deverá receber expressões como `40+2` ou `84 / 2` e retornar o resultado ao cliente.

3.1 Análise dos Recursos

Detecção do *Ctrl + C* no servidor:



The image shows a terminal window titled "Tilix: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/echo". The terminal displays the following sequence of commands and output:

```
1: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/echo
(pm) debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$ python3 servidor.py
[-] Servidor rodando em 127.0.0.1:5000... Pressione Ctrl+C para interromper.
^C
[*] Ctrl+C detectado! Desligando o servidor com segurança...
[*] Porta liberada. Finalizado.
(pm) debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$
```

O comando `netstat` pode ser usado, tanto no Windows quanto no Linux / macOS para verificar os *sockets*:

- Usar o `sudo` antes do `netstat` o executa com privilégios de superusuário, permitindo que a ferramenta acesse a memória e as *interfaces* do *kernel*, listando absolutamente todas as conexões e processos do sistema
- `-t`: Lista apenas as conexões que utilizam o protocolo TCP
- `-u`: Filtra somente o protocolo UDP
- `-l`: Mostra apenas as conexões que estão abertas e aguardando conexões
- `-p`: Exibe o PID (ID do Processo) e o nome do programa responsável por abrir aquela porta (exige `sudo`)
- `-n`: Exibe os endereços IP e os números das portas em formato numérico
- `-a`: Mostra tanto as portas em escuta quanto as conexões estabelecidas

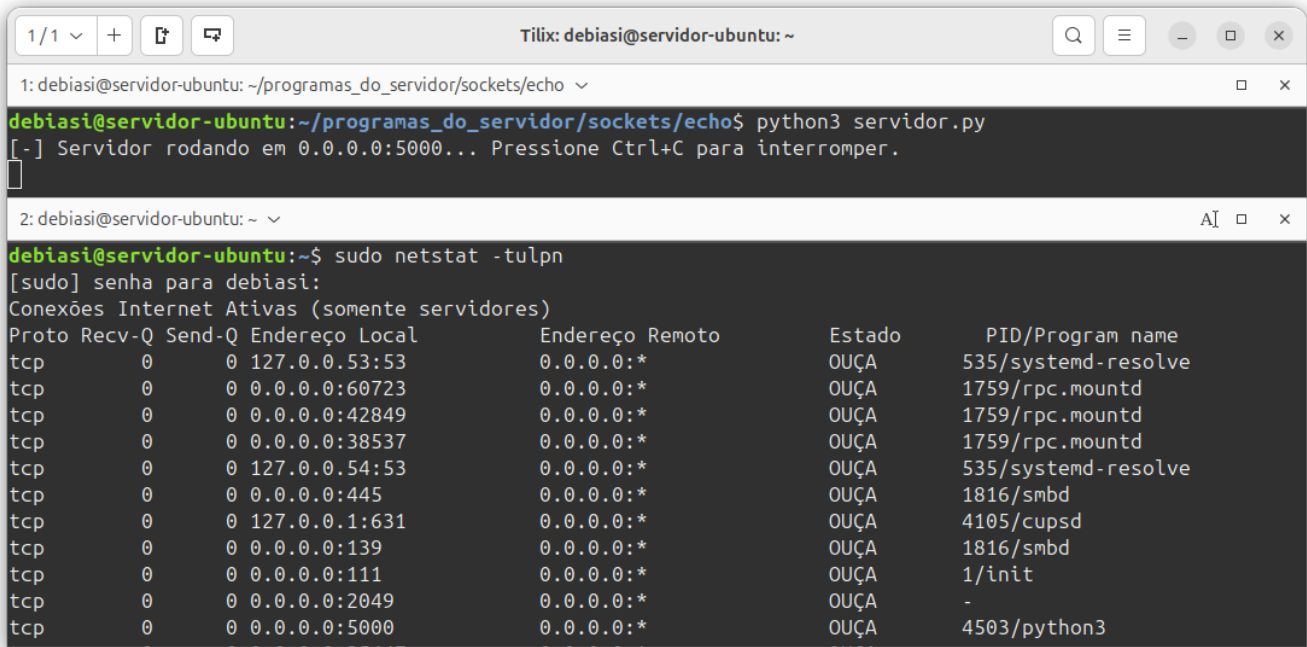
Exemplos:

```
➤ sudo netstat -a
```

```
1/1  +  [ ] [ ]
Tilix: debiasi@servidor-ubuntu: ~
1: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/echo ~
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$ python3 servidor.py
[-] Servidor rodando em 0.0.0.0:5000... Pressione Ctrl+C para interromper.

2: debiasi@servidor-ubuntu: ~
debiasi@servidor-ubuntu:~$ sudo netstat -a
Conexões Internet Ativas (servidores e estabelecidas)
Proto Recv-Q Send-Q Endereço Local          Endereço Remoto          Estado
tcp        0      0 _localdnstStub:domain   0.0.0.0:*                OUÇA
tcp        0      0 0.0.0.0:60723           0.0.0.0:*                OUÇA
tcp        0      0 0.0.0.0:42849           0.0.0.0:*                OUÇA
tcp        0      0 0.0.0.0:38537           0.0.0.0:*                OUÇA
tcp        0      0 _localdnstProxy:domain  0.0.0.0:*                OUÇA
tcp        0      0 0.0.0.0:microsoft-ds    0.0.0.0:*                OUÇA
tcp        0      0 localhost:ipp            0.0.0.0:*                OUÇA
tcp        0      0 0.0.0.0:netbios-ssn     0.0.0.0:*                OUÇA
tcp        0      0 0.0.0.0:sunrpc           0.0.0.0:*                OUÇA
tcp        0      0 0.0.0.0:nfs              0.0.0.0:*                OUÇA
tcp        0      0 0.0.0.0:5000              0.0.0.0:*                OUÇA
tcp        0      0 0.0.0.0:35447            0.0.0.0:*                OUÇA
tcp        0      0 0.0.0.0:37393            0.0.0.0:*                OUÇA
tcp        0      0 192.168.20.10:33274     142.251.150.119:https    ESTABELECIDA
tcp        0      0 192.168.20.10:41454     93.243.107.34.bc.:https  ESTABELECIDA
tcp6       0      0 ip6-localhost:ipp      [::]:*                  OUÇA
tcp6       0      0 [::]:3317              [::]:*                  OUÇA
```

➤ `sudo netstat -tulpn`



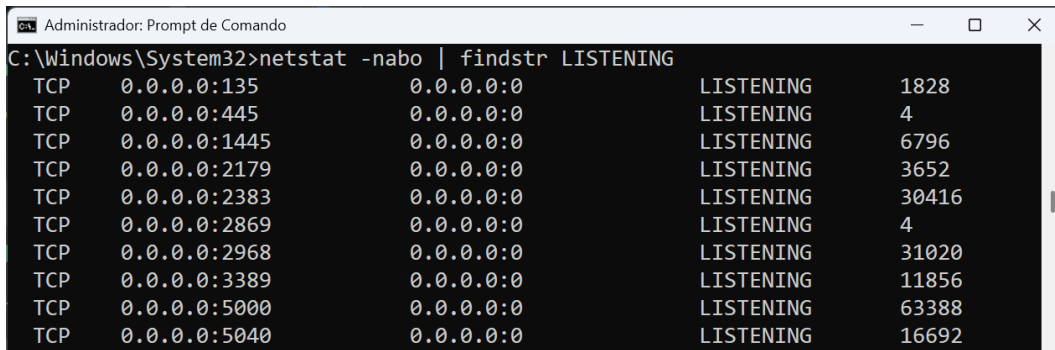
The terminal shows two sessions. Session 1: A python3 server is running in the background, outputting 'Servidor rodando em 0.0.0.0:5000... Pressione Ctrl+C para interromper.' Session 2: The user runs 'sudo netstat -tulpn', which prompts for a password and then displays a table of active internet connections.

```
1: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/echo ~
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$ python3 servidor.py
[-] Servidor rodando em 0.0.0.0:5000... Pressione Ctrl+C para interromper.

2: debiasi@servidor-ubuntu: ~
debiasi@servidor-ubuntu:~$ sudo netstat -tulpn
[sudo] senha para debiasi:
Conexões Internet Ativas (somente servidores)
Proto Recv-Q Send-Q Endereço Local          Endereço Remoto          Estado      PID/Program name
tcp      0      0 127.0.0.53:53           0.0.0.0:*                 OUÇA       535/systemd-resolve
tcp      0      0 0.0.0.0:60723          0.0.0.0:*                 OUÇA       1759/rpc.mountd
tcp      0      0 0.0.0.0:42849          0.0.0.0:*                 OUÇA       1759/rpc.mountd
tcp      0      0 0.0.0.0:38537          0.0.0.0:*                 OUÇA       1759/rpc.mountd
tcp      0      0 127.0.0.54:53          0.0.0.0:*                 OUÇA       535/systemd-resolve
tcp      0      0 0.0.0.0:445            0.0.0.0:*                 OUÇA       1816/smbd
tcp      0      0 127.0.0.1:631          0.0.0.0:*                 OUÇA       4105/cupsd
tcp      0      0 0.0.0.0:139            0.0.0.0:*                 OUÇA       1816/smbd
tcp      0      0 0.0.0.0:111            0.0.0.0:*                 OUÇA       1/init
tcp      0      0 0.0.0.0:2049            0.0.0.0:*                 OUÇA       -
tcp      0      0 0.0.0.0:5000            0.0.0.0:*                 OUÇA       4503/python3
```

No Windows deve-se abrir o CMD em modo administrador:

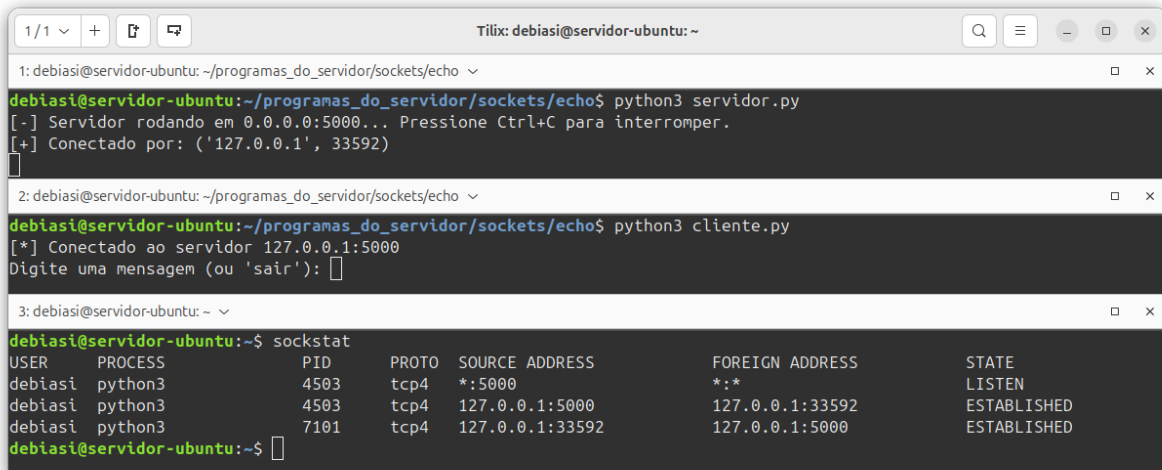
- -a: Mostra todas as conexões e portas
 - -n: Exibe números de IPs e portas em formato numérico (mais rápido pois não faz resolução de nomes)
 - -o: Exibe o PID (ID do processo) dono daquela conexão
- `findstr`: Filtra os resultados, mostrando somente os *sockets* aguardando conexão
- `netstat -nabo | findstr LISTENING`



The Windows Command Prompt shows the command 'netstat -nabo | findstr LISTENING' being executed. The output is a table of listening ports and their associated PIDs.

```
C:\Windows\System32>netstat -nabo | findstr LISTENING
TCP      0.0.0.0:135             0.0.0.0:0                 LISTENING   1828
TCP      0.0.0.0:445             0.0.0.0:0                 LISTENING    4
TCP      0.0.0.0:1445            0.0.0.0:0                 LISTENING  6796
TCP      0.0.0.0:2179            0.0.0.0:0                 LISTENING  3652
TCP      0.0.0.0:2383            0.0.0.0:0                 LISTENING 30416
TCP      0.0.0.0:2869            0.0.0.0:0                 LISTENING    4
TCP      0.0.0.0:2968            0.0.0.0:0                 LISTENING 31020
TCP      0.0.0.0:3389            0.0.0.0:0                 LISTENING 11856
TCP      0.0.0.0:5000            0.0.0.0:0                 LISTENING 63388
TCP      0.0.0.0:5040            0.0.0.0:0                 LISTENING 16692
```

Exemplo do comando `sockstat`, nativo do FreeBSD (`sudo apt install sockstat`):



The terminal shows three sessions. Session 1: The python3 server is running. Session 2: A python3 client connects to the server at 127.0.0.1:5000. Session 3: The user runs 'sockstat', which displays a table of active sockets.

```
1: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/echo ~
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$ python3 servidor.py
[-] Servidor rodando em 0.0.0.0:5000... Pressione Ctrl+C para interromper.
[+] Conectado por: ('127.0.0.1', 33592)

2: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/echo ~
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$ python3 cliente.py
[*] Conectado ao servidor 127.0.0.1:5000
Digite uma mensagem (ou 'sair'):

3: debiasi@servidor-ubuntu: ~
debiasi@servidor-ubuntu:~$ sockstat
USER  PROCESS  PID  PROTO  SOURCE ADDRESS  FOREIGN ADDRESS  STATE
debiasi python3 4503 tcp4   *:5000          *:~              LISTEN
debiasi python3 4503 tcp4   127.0.0.1:5000 127.0.0.1:33592 ESTABLISHED
debiasi python3 7101 tcp4   127.0.0.1:33592 127.0.0.1:5000  ESTABLISHED
```

O **ss** (**Socket Statistics**) é o substituto moderno do **netstat** e alternativa nativa do Linux ao **sockstat**. O **ss** conversa diretamente com o *kernel* do Linux utilizando uma API chamada **Netlink**.

```
1/1 v + [?] [?]
Tili: debiasi@servidor-ubuntu: ~

1: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/echo v
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$ python3 servidor.py
[-] Servidor rodando em 0.0.0.0:5000... Pressione Ctrl+C para interromper.
[+] Conectado por: ('127.0.0.1', 33592)

2: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/echo v
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$ python3 cliente.py
[*] Conectado ao servidor 127.0.0.1:5000
Digite uma mensagem (ou 'sair'):
```

```
3: debiasi@servidor-ubuntu: ~ v
debiasi@servidor-ubuntu:~$ sudo ss -tulwn
Netid      State      Recv-Q     Send-Q       Local Address:Port      Peer Address:Port      Process
tcp6       UNCONN     0           0           *:58                    *:58                    users:((("NetworkManager",pid=1304,fd=29))
udp        UNCONN     0           0           0.0.0.0:34253          0.0.0.0:*              users:((("systemd-resolve",pid=535,fd=16))
udp        UNCONN     0           0           127.0.0.53%lo:53       0.0.0.0:*              users:((("systemd-resolve",pid=535,fd=14))
udp        UNCONN     0           0           0.0.0.0:111           0.0.0.0:*              users:((("rpcbind",pid=509,fd=5),("systemd",pid=1,fd=428))
udp        UNCONN     0           0           192.168.20.255:137     0.0.0.0:*              users:((("nmbd",pid=1792,fd=16))
udp        UNCONN     0           0           192.168.20.10:137     0.0.0.0:*              users:((("nmbd",pid=1792,fd=15))
udp        UNCONN     0           0           0.0.0.0:137           0.0.0.0:*              users:((("nmbd",pid=1792,fd=13))
udp        UNCONN     0           0           192.168.20.255:138     0.0.0.0:*              users:((("nmbd",pid=1792,fd=18))
udp        UNCONN     0           0           192.168.20.10:138     0.0.0.0:*              users:((("nmbd",pid=1792,fd=17))
udp        UNCONN     0           0           0.0.0.0:138           0.0.0.0:*              users:((("nmbd",pid=1792,fd=14))
udp        UNCONN     0           0           0.0.0.0:47254          0.0.0.0:*              users:((("rpc.mountd",pid=1759,fd=4))
udp        UNCONN     0           0           0.0.0.0:47715          0.0.0.0:*              users:((("rpc.mountd",pid=1759,fd=8))
udp        UNCONN     0           0           127.0.0.1:671         0.0.0.0:*              users:((("rpc.statd",pid=1767,fd=5))
udp        UNCONN     0           0           0.0.0.0:54128          0.0.0.0:*              users:((("avahi-daemon",pid=1196,fd=14))
udp        UNCONN     0           0           0.0.0.0:42196          0.0.0.0:*              users:((("rpc.statd",pid=1767,fd=8))
udp        UNCONN     0           0           0.0.0.0:5353           0.0.0.0:*              users:((("avahi-daemon",pid=1196,fd=12))
udp        UNCONN     0           0           0.0.0.0:36074          0.0.0.0:*              users:((("rpc.mountd",pid=1759,fd=12))
udp        UNCONN     0           0           [::]:52493            [::]:*                 users:((("rpc.mountd",pid=1759,fd=14))
udp        UNCONN     0           0           [::]:44331            [::]:*                 users:((("rpc.statd",pid=1767,fd=10))
udp        UNCONN     0           0           [::]:50765            [::]:*                 users:((("rpc.mountd",pid=1759,fd=6))
udp        UNCONN     0           0           [::]:53321            [::]:*                 users:((("rpcbind",pid=509,fd=7),("systemd",pid=1,fd=430))
udp        UNCONN     0           0           [::]:111              [::]:*                 users:((("rpc.mountd",pid=1759,fd=10))
udp        UNCONN     0           0           [::]:32940            [::]:*                 users:((("rpc.mountd",pid=1759,fd=10))
udp        UNCONN     0           0           [::]:41558            [::]:*                 users:((("avahi-daemon",pid=1196,fd=15))
udp        UNCONN     0           0           [::]:5353             [::]:*                 users:((("avahi-daemon",pid=1196,fd=13))
tcp6       LISTEN     0           4096        127.0.0.1:5000         0.0.0.0:*              users:((("systemd-resolve",pid=535,fd=16))
```

Outros exemplos:

- **-a**: Mostra todas as conexões e portas
 - **-n**: Exibe números de IPs e portas em formato numérico (mais rápido pois não faz resolução de nomes)
 - **-w (RAW)**: Exibe os **sockets** do tipo RAW (soquetes brutos), que permitem que o aplicativo envie e receba pacotes IP diretamente, pulando as camadas tradicionais de transporte (TCP/UDP) – são muito usados por ferramentas de diagnóstico de rede (ping, traceroute ou farejadores de pacotes como o tcpdump)
 - **sport = :5000**: Filtra pelo número da porta de origem (**dport** filtra pela porta de destino)
 - **column**: Aumenta o espaçamento entre as larguras na saída do comando **ss**
 - **|**: O **pipe** é um mecanismo IPC (comunicação inter-processos) que conecta, neste caso, o **ss** com o **column**
 - **-t state <estado>**: Filtra por estado da conexão, como **established**, **syn-sent**, **time-wait**, etc.
- **ss -t state established**
- **ss -tulpn sport = :5000 | column -t -o ' ' '**

```
1/1 v + [?] [?]
Tili: debiasi@servidor-ubuntu: ~

1: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/echo v
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$ python3 servidor.py
[-] Servidor rodando em 0.0.0.0:5000... Pressione Ctrl+C para interromper.
[+] Conectado por: ('127.0.0.1', 33592)

2: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/echo v
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$ python3 cliente.py
[*] Conectado ao servidor 127.0.0.1:5000
Digite uma mensagem (ou 'sair'):
```

```
3: debiasi@servidor-ubuntu: ~ v
debiasi@servidor-ubuntu:~$ ss -t state established
Recv-Q     Send-Q       Local Address:Port      Peer Address:Port      Process
0           0           127.0.0.1:33592         127.0.0.1:5000
0           0           127.0.0.1:5000         127.0.0.1:33592

debiasi@servidor-ubuntu:~$
debiasi@servidor-ubuntu:~$ ss -tulpn sport = :5000 | column -t -o ' '
Netid      State      Recv-Q     Send-Q       Local Address:Port      Peer Address:Port      Process
tcp        LISTEN     0           1           0.0.0.0:5000           0.0.0.0:*              users:((("python3",pid=4503,fd=3))
```

O comando `lsof` (*List Open Files*) trata *sockets* de rede como arquivos abertos. Ele possui uma saída muito organizada e exibe o nome do usuário do sistema que rodou aquele comando (coluna `USER`).

- `grep`: O *pipe* (`|`) envia a saída do `lsof` para o comando `grep`, que filtra as linhas que contenham a palavra **LISTEN** **OU** as linhas que comecem com a palavra `COMMAND` (que é o início do cabeçalho do `lsof`).

```
1/1 ~ + [T] [C] TiliX: debiasi@servidor-ubuntu: ~
1: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/echo ~
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$ python3 servidor.py
[-] Servidor rodando em 0.0.0.0:5000... Pressione Ctrl+C para interromper.
[+] Conectado por: ('127.0.0.1', 33592)

2: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/echo ~
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$ python3 cliente.py
[*] Conectado ao servidor 127.0.0.1:5000
Digite uma mensagem (ou 'sair'):

3: debiasi@servidor-ubuntu: ~ ~
debiasi@servidor-ubuntu:~$ sudo lsof -i -P -n | grep -E "COMMAND|LISTEN"
COMMAND PID USER FD TYPE DEVICE SIZE/OFF NODE NAME
systemd 1 root 427u IPv4 11207 0t0 TCP *:111 (LISTEN)
systemd 1 root 429u IPv6 11209 0t0 TCP *:111 (LISTEN)
rpcbind 509 _rpc 4u IPv4 11207 0t0 TCP *:111 (LISTEN)
rpcbind 509 _rpc 6u IPv6 11209 0t0 TCP *:111 (LISTEN)
systemd-r 535 systemd-resolve 15u IPv4 22732 0t0 TCP 127.0.0.53:53 (LISTEN)
systemd-r 535 systemd-resolve 17u IPv4 22734 0t0 TCP 127.0.0.54:53 (LISTEN)
rpc.mount 1759 root 5u IPv4 27971 0t0 TCP *:38537 (LISTEN)
rpc.mount 1759 root 7u IPv6 27987 0t0 TCP *:49235 (LISTEN)
rpc.mount 1759 root 9u IPv4 27999 0t0 TCP *:60723 (LISTEN)
rpc.mount 1759 root 11u IPv6 28011 0t0 TCP *:59503 (LISTEN)
rpc.mount 1759 root 13u IPv4 28023 0t0 TCP *:42849 (LISTEN)
rpc.mount 1759 root 15u IPv6 28035 0t0 TCP *:33177 (LISTEN)
rpc.statd 1767 statd 9u IPv4 26498 0t0 TCP *:37393 (LISTEN)
rpc.statd 1767 statd 11u IPv6 26504 0t0 TCP *:33423 (LISTEN)
smbd 1816 root 28u IPv6 28371 0t0 TCP *:445 (LISTEN)
smbd 1816 root 29u IPv6 28372 0t0 TCP *:139 (LISTEN)
smbd 1816 root 30u IPv4 28373 0t0 TCP *:445 (LISTEN)
smbd 1816 root 31u IPv4 28374 0t0 TCP *:139 (LISTEN)
cupsd 4105 root 6u IPv6 217609 0t0 TCP [::]:631 (LISTEN)
cupsd 4105 root 7u IPv4 217610 0t0 TCP 127.0.0.1:631 (LISTEN)
python3 4503 debiasi 3u IPv4 219436 0t0 TCP *:5000 (LISTEN)
debiasi@servidor-ubuntu:~$
```

Verificando o processo servidor através dos comandos `netstat`/`ss`/`lsof` recurso sendo utilizado (porta) e o finalizando ('matando') com o comando `kill`:

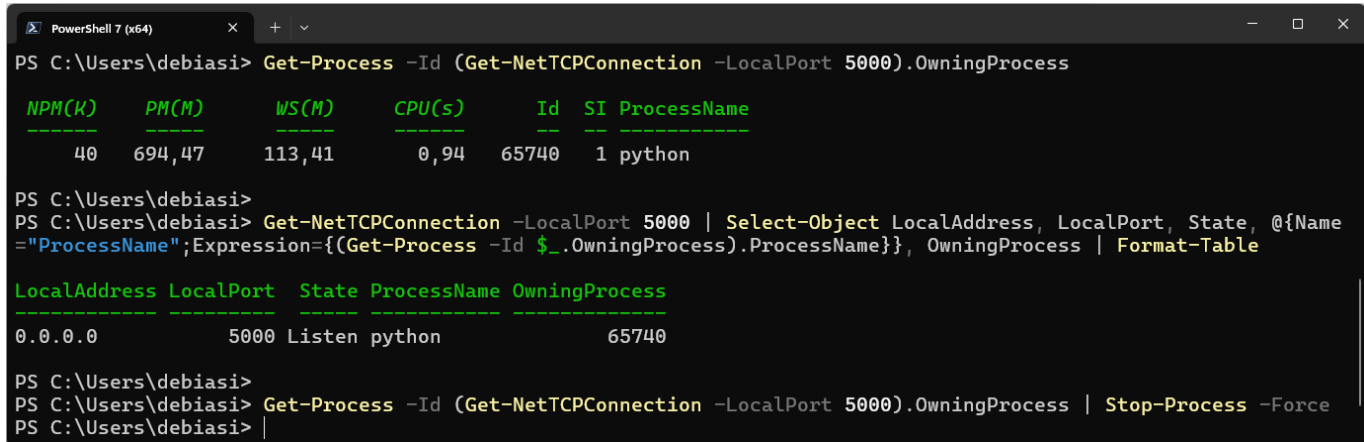
- `lsof -i :5000`
- `kill -9 $(lsof -t -i :5000)`

```
1/1 ~ + [T] [C] TiliX: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/echo
1: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/echo ~
(pm) debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$ python3 servidor.py
[-] Servidor rodando em 127.0.0.1:5000... Pressione Ctrl+C para interromper.
Morto
(pm) debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$

2: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/echo ~
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$ netstat -tlnp | grep :5000
(Nem todos os processos puderam ser identificados, informações sobre processos
de outrem não serão mostrados, você deve ser root para vê-los todos.)
tcp 0 0 127.0.0.1:5000 0.0.0.0:* OÚÇA 5249/python3
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$ ss -tulnp | grep 5000
tcp LISTEN 0 1 127.0.0.1:5000 0.0.0.0:* users:(("python3",pid=5249,fd=3))
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$ lsof -i :5000
COMMAND PID USER FD TYPE DEVICE SIZE/OFF NODE NAME
python3 5249 debiasi 3u IPv4 74130 0t0 TCP localhost:5000 (LISTEN)
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$ kill -9 $(lsof -t -i :5000)
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$
```

No Windows, identificar e finalizar o processo que criou uma porta específica poderia ser feito da seguinte forma no PowerShell:

- `Get-Process -Id (Get-NetTCPConnection -LocalPort 5000).OwningProcess`
- `Get-NetTCPConnection -LocalPort 5000 | Select-Object LocalAddress, LocalPort, State, @{Name="ProcessName"; Expression={(Get-Process -Id $_.OwningProcess).ProcessName}}, OwningProcess | Format-Table`
- `Get-Process -Id (Get-NetTCPConnection -LocalPort 5000).OwningProcess | Stop-Process -Force`



```
PS C:\Users\debiasi> Get-Process -Id (Get-NetTCPConnection -LocalPort 5000).OwningProcess
```

NPM(K)	PM(M)	WS(M)	CPU(s)	Id	SI	ProcessName
40	694,47	113,41	0,94	65740	1	python

```
PS C:\Users\debiasi> Get-NetTCPConnection -LocalPort 5000 | Select-Object LocalAddress, LocalPort, State, @{Name="ProcessName"; Expression={(Get-Process -Id $_.OwningProcess).ProcessName}}, OwningProcess | Format-Table
```

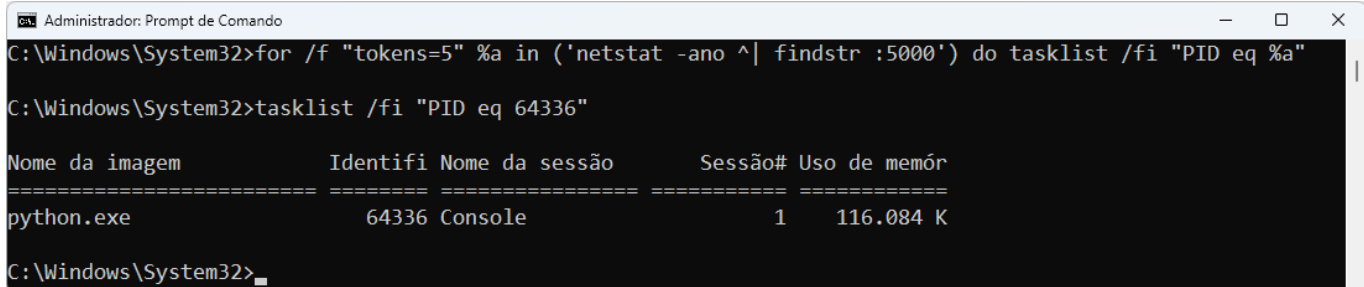
LocalAddress	LocalPort	State	ProcessName	OwningProcess
0.0.0.0	5000	Listen	python	65740

```
PS C:\Users\debiasi> Get-Process -Id (Get-NetTCPConnection -LocalPort 5000).OwningProcess | Stop-Process -Force
PS C:\Users\debiasi>
```

No CMD não existe uma forma simples de fazer isso porque a versão do `netstat` que mostra o PID dono do `socket` divide a saída em duas linhas, o que invalida o filtro com o comando `findstr`.

Como o CMD não possui substituição direta de comandos como o Linux, é preciso usar um laço `FOR` para capturar o PID gerado pelo `netstat` e passá-lo para o comando `taskkill` (o equivalente ao `kill -9` do Windows):

- `for /f "tokens=5" %a in ('netstat -ano ^| findstr :5000') do tasklist /fi "PID eq %a"`



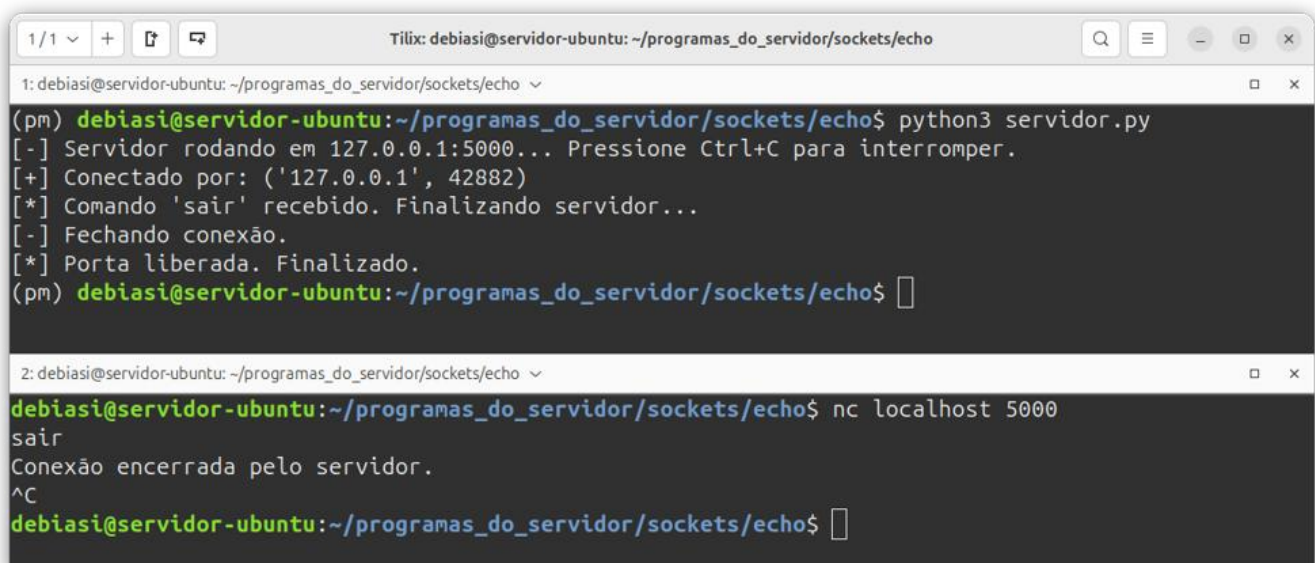
```
C:\Windows\System32>for /f "tokens=5" %a in ('netstat -ano ^| findstr :5000') do tasklist /fi "PID eq %a"

C:\Windows\System32>tasklist /fi "PID eq 64336"

Nome da imagem                Identifi Nome da sessão      Sessão#  Uso de memór
=====
python.exe                    64336 Console                  1      116.084 K

C:\Windows\System32>
```

Usando o `nc` (netcat, canivete suíço do Linux) no cliente para conectar com o cliente:



```
Tilix: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/echo

1: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/echo ~
(pm) debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$ python3 servidor.py
[-] Servidor rodando em 127.0.0.1:5000... Pressione Ctrl+C para interromper.
[+] Conectado por: ('127.0.0.1', 42882)
[*] Comando 'sair' recebido. Finalizando servidor...
[-] Fechando conexão.
[*] Porta liberada. Finalizado.
(pm) debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$

2: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/echo ~
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$ nc localhost 5000
sair
Conexão encerrada pelo servidor.
^C
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/echo$
```

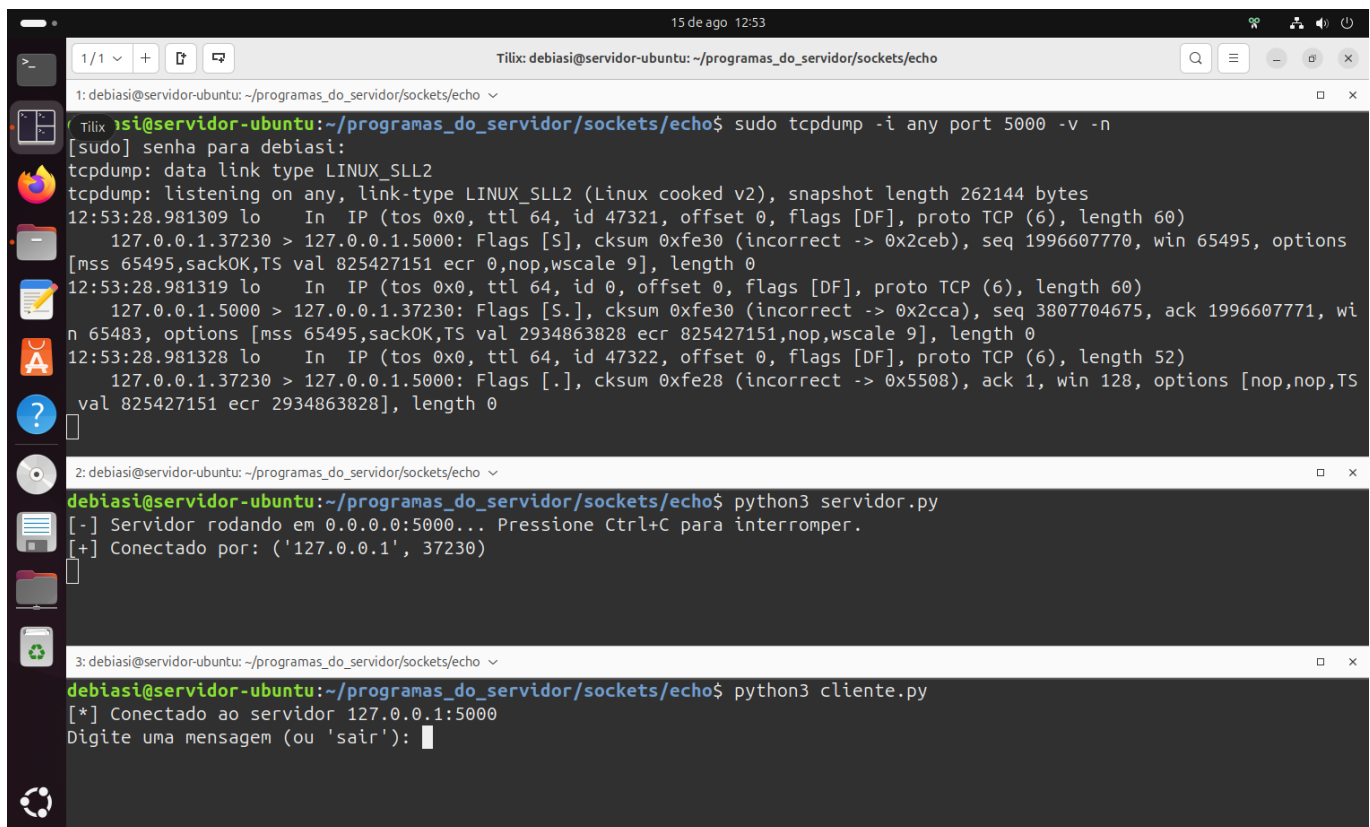
3.2 Análise com tcpdump

Para analisar a **Three-Way Handshake (SYN, SYN-ACK, ACK)** usando o `tcpdump`, é preciso capturar os pacotes especificamente na *interface* de rede e na porta que estão sendo utilizadas.

```
➤ sudo tcpdump -i any port 50000 -v -n
```

Parâmetros:

- `-i any`: Escuta em **todas** as *interfaces* de rede (cabo, Wi-Fi e *localhost/loopback*). Se estiver se testando na mesma máquina, o `any` ou `lo` irá capturar o tráfego local.
- `port 5000`: Filtra o tráfego. O `tcpdump` só vai mostrar os pacotes que entrarem ou saírem pela porta do seu 'Servidor Echo'.
- `-v`: Ativa o modo detalhado (*verbose*), mostrando informações como o TTL e o tamanho do cabeçalho.
- `-n`: Não resolve IPs para nomes de domínio (exibe `192.168.0.100` em vez de tentar adivinhar o nome da máquina). Deixa a captura muito mais rápida



```
15 de ago 12:53
Tilix: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/echo
1: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/echo
$ sudo tcpdump -i any port 5000 -v -n
[sudo] senha para debiasi:
tcpdump: data link type LINUX_SLL2
tcpdump: listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
12:53:28.981309 lo In IP (tos 0x0, ttl 64, id 47321, offset 0, flags [DF], proto TCP (6), length 60)
127.0.0.1.37230 > 127.0.0.1.5000: Flags [S], cksum 0xfe30 (incorrect -> 0x2ceb), seq 1996607770, win 65495, options
[mss 65495,sackOK,TS val 825427151 ecr 0,nop,wscale 9], length 0
12:53:28.981319 lo In IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 60)
127.0.0.1.5000 > 127.0.0.1.37230: Flags [S.], cksum 0xfe30 (incorrect -> 0x2cca), seq 3807704675, ack 1996607771, wi
n 65483, options [mss 65495,sackOK,TS val 2934863828 ecr 825427151,nop,wscale 9], length 0
12:53:28.981328 lo In IP (tos 0x0, ttl 64, id 47322, offset 0, flags [DF], proto TCP (6), length 52)
127.0.0.1.37230 > 127.0.0.1.5000: Flags [.], cksum 0xfe28 (incorrect -> 0x5508), ack 1, win 128, options [nop,nop,TS
val 825427151 ecr 2934863828], length 0

2: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/echo
$ python3 servidor.py
[-] Servidor rodando em 0.0.0.0:5000... Pressione Ctrl+C para interromper.
[+] Conectado por: ('127.0.0.1', 37230)

3: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/echo
$ python3 cliente.py
[*] Conectado ao servidor 127.0.0.1:5000
Digite uma mensagem (ou 'sair'): 
```

Analisando a resposta:

1. O primeiro pacote (*SYN: Cliente → Servidor*): O cliente (porta **37230**) envia uma solicitação para a porta do servidor (**5000**). A *flag* **[S]** confirma que este é o pacote **SYN**. Note que o cliente define o seu número de sequência inicial como **1996607770**. *Timestamp (HORA:MINUTO:SEGUNDO.MICRO_SEGUNDOS)* é **12:53:28.981309**.
2. O segundo pacote (*SYN-ACK: Servidor → Cliente*): Apenas **10** microssegundos depois (*timestamp* **12:53:28.981319**), o servidor (porta **5000**) responde ao cliente (porta **37230**). A *flag* **[S.]** indica **SYN-ACK**. O servidor cria o seu próprio número de sequência (**3807704675**). O campo **ack 1996607771** prova que o servidor recebeu o SYN do cliente (pois pegou o número de sequência do cliente e somou +1).
3. O terceiro pacote (*ACK: Cliente → Servidor*): Mais **9 us** depois (**12:53:28.981328**), o cliente responde de volta ao servidor para finalizar a abertura. A *flag* **[.]** indica o **ACK** final. O **ack 1** (aqui simplificado pelo `tcpdump` como número relativo) avisa ao servidor que o seu **SYN-ACK** foi recebido com sucesso.

3.3 Análise com Wireshark

Análise com o Wireshark:

➤ `sudo apt install wireshark`



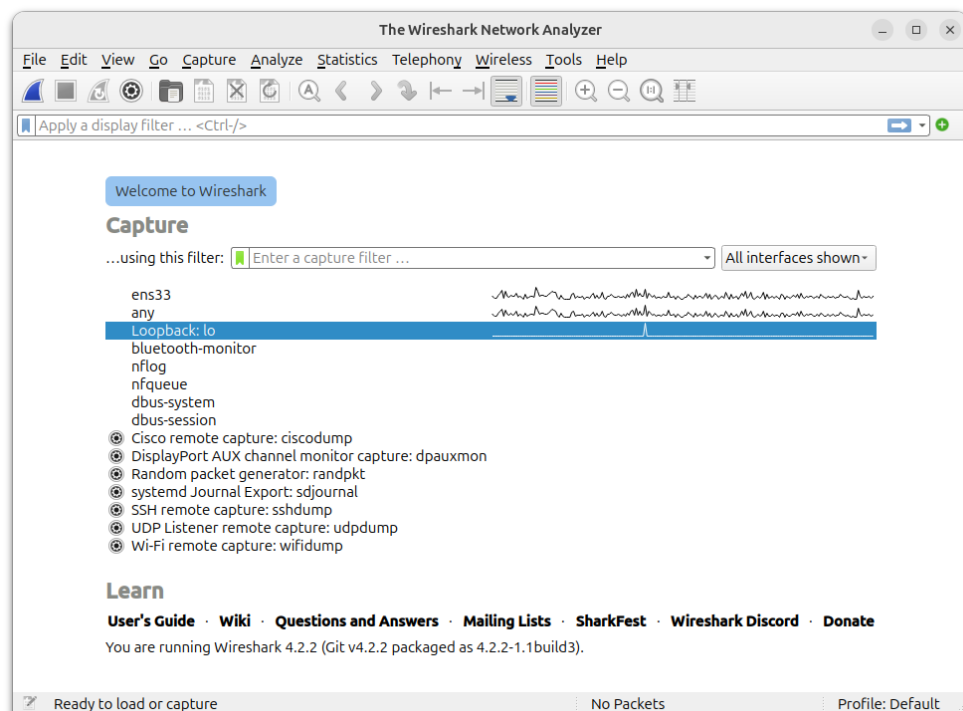
Abra o terminal e execute o comando abaixo:

- `sudo usermod -a -G wireshark $USER`
- `exec su - $USER`
- `sudo chmod 4711 `sudo which dumpcap``
- `groups`

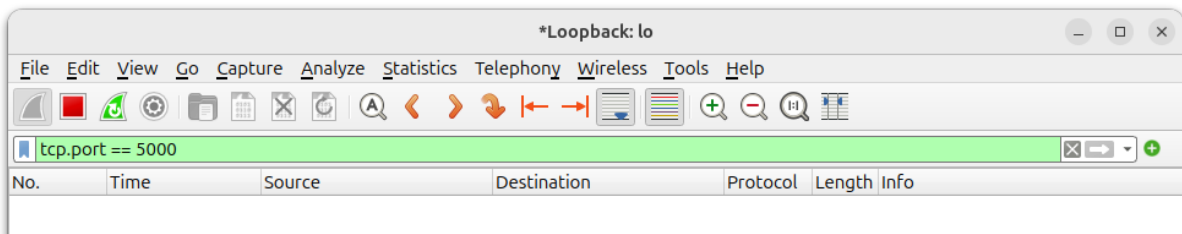
Verifique se o grupo `wireshark` aparece na lista, caso não apareça, reinicie o Ubuntu.

Fazendo a verificação:

1. Abra o Wireshark através da *interface* gráfica e dê um clique duplo na *interface loopback* para iniciar a captura



2. Na tela de filtro insira `tcp.port == 5000` e dê um *Enter*

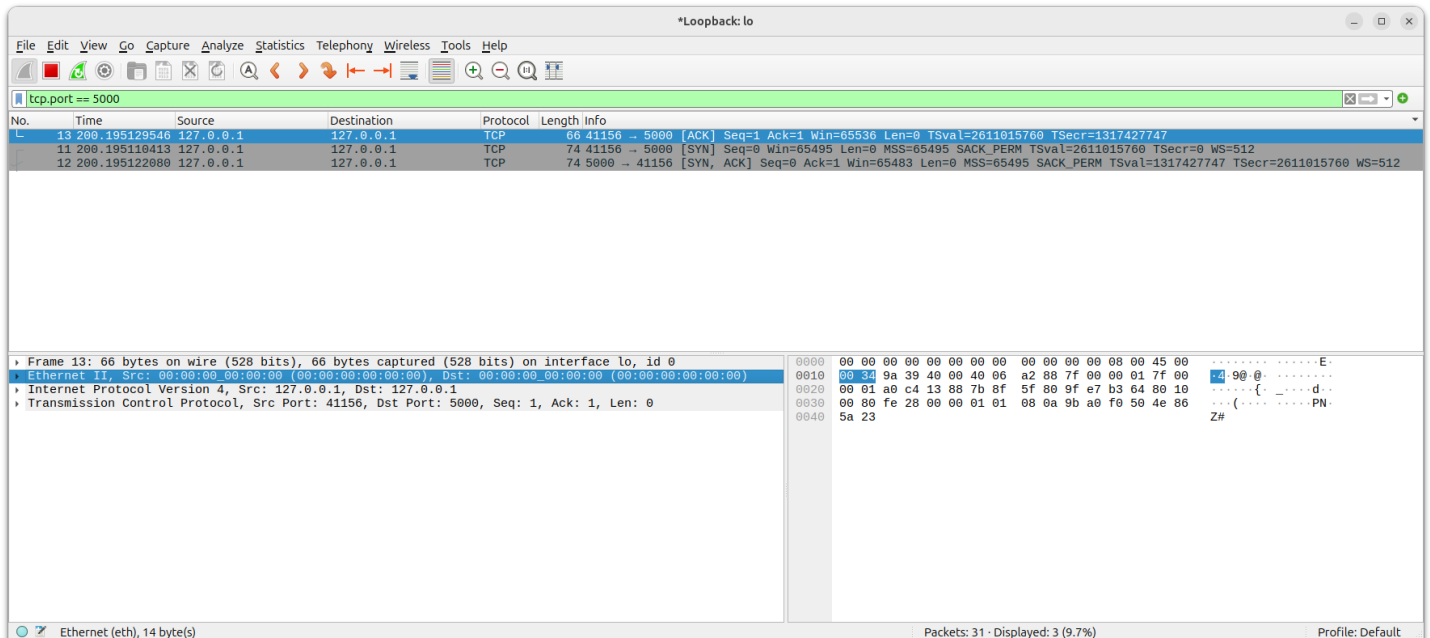


3. Abra o Tilix e divida a tela em duas, na horizontal.

4. Na tela superior execute o servidor `echo` e na de baixo, o cliente



5. Observe a tela do Wireshark

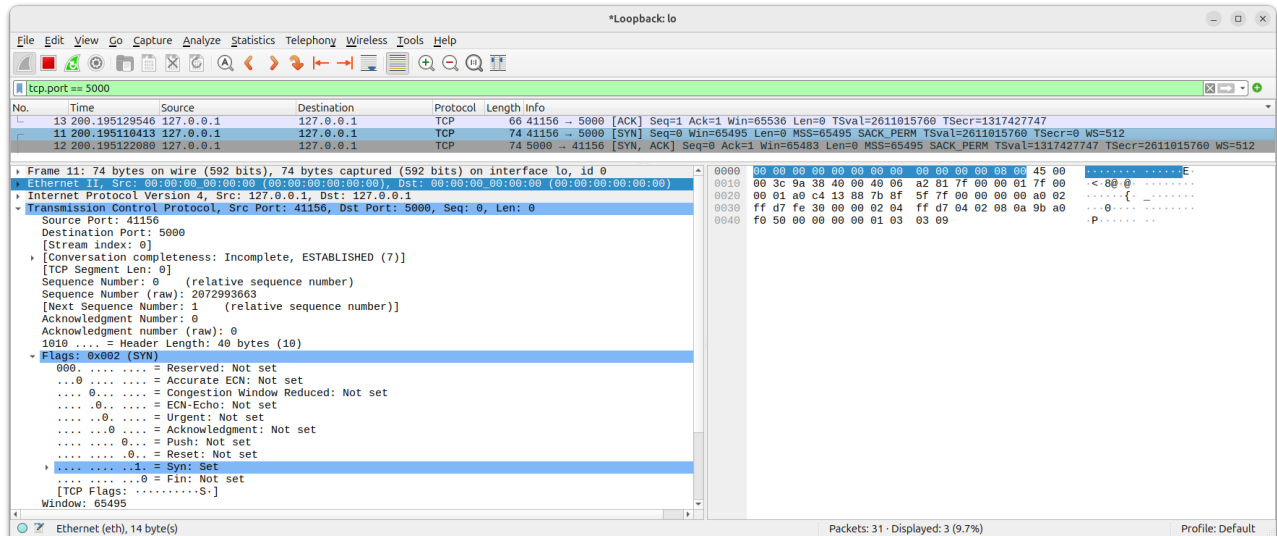


Na janela do Wireshark, você verá uma sequência de pacotes. Procure pelas três primeiras linhas na coluna *Info*:

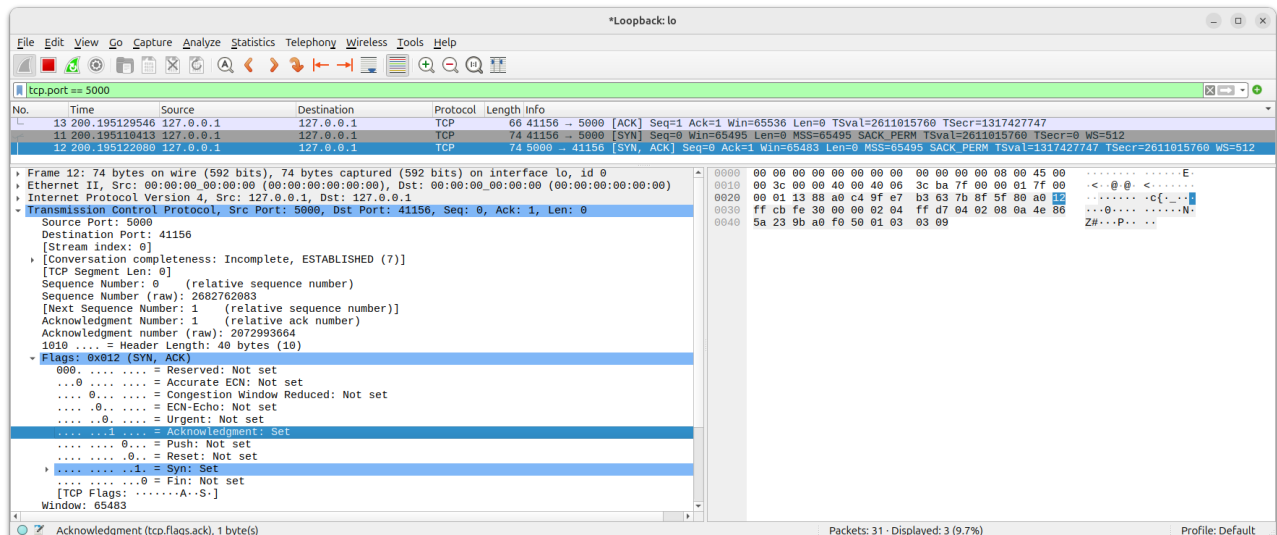
- [SYN]: Enviado pelo cliente para iniciar a conexão.
- [SYN, ACK]: Respondido pelo servidor aceitando a conexão.
- [ACK]: Enviado pelo cliente confirmando que a conexão foi estabelecida.

Analisando os detalhes:

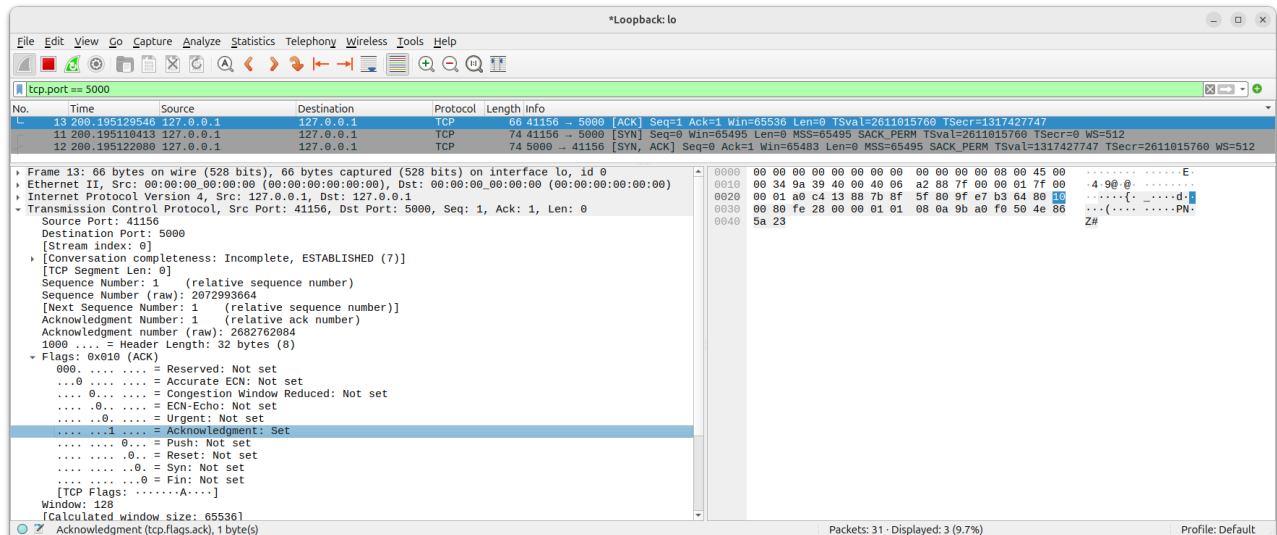
- Selecione a primeira linha e abra os detalhes do TCP, verifique que o bit [SYN], enviado pelo cliente para iniciar a conexão, está setado



- Clique agora na segunda linha – o detalhe mostra que agora os bits [SYN] e [ACK], respondido pelo servidor aceitando a conexão estão setados



- A terceira linha mostra o bit [ACK] setado, enviado pelo cliente confirmando que a conexão foi estabelecida



4 Servidor e Cliente UDP

O UDP é ideal para se perceber a diferença entre protocolos orientados a conexão (TCP) e não-orientados (UDP).

Com o UDP (*User Datagram Protocol*), não existe `accept()` nem `connect()`. O servidor apenas escuta, e quem envia precisa especificar o endereço a cada pacote. Enquanto o TCP cria a ilusão de um ‘circuito fechado’ ou ‘conexão direta’ entre duas máquinas, o UDP funciona exatamente como a internet real opera na sua camada mais básica: enviando pacotes isolados.

Servidor:

```
import socket
import sys

HOST = sys.argv[1] if len(sys.argv) > 1 else '0.0.0.0'
PORT = int(sys.argv[2]) if len(sys.argv) > 2 else 6000

# SOCK_DGRAM define que o protocolo é UDP
server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind((HOST, PORT))

print("[+] Servidor UDP pronto na porta 6000...")

while True:
    # recvfrom retorna os dados e o endereço (IP, Porta) de quem enviou
    dados, endereço_cliente = server.recvfrom(1024)
    mensagem = dados.decode('utf-8')
    print(f"[{endereço_cliente}]: {mensagem}")

    # Resposta direta para o endereço que acabou de enviar
    resposta = f"UDP Echo: {mensagem.upper()}"
    server.sendto(resposta.encode('utf-8'), endereço_cliente)
```

Cliente:

```
import socket
import sys

HOST = '127.0.0.1'
PORT = 6000

if len(sys.argv) > 1:
    HOST = sys.argv[1]

if len(sys.argv) > 2:
    try:
        PORT = int(sys.argv[2])
    except ValueError:
        print("Erro: A porta deve ser um número inteiro.")
        sys.exit(1)

client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
destino = (HOST, PORT)

mensagem = "Testando conexao UDP"
# No UDP, enviamos direto para o destino sem precisar de connect()
client.sendto(mensagem.encode('utf-8'), destino)

# Recebe a resposta e o endereço do servidor
dados, servidor = client.recvfrom(1024)
print(f"[Servidor]: {dados.decode('utf-8')}")

client.close()
```

No modelo TCP, o estabelecimento de uma conexão exige uma dança de funções (o famoso *three-way handshake*):

- No **servidor**: O código chama `listen()` e depois trava em `accept()`, esperando um cliente bater à porta. Quando o cliente conecta, o `accept()` cria um novo *socket* dedicado exclusivamente para conversar com aquele cliente específico.
- No **cliente**: O código chama `connect()`, que envia uma solicitação de sincronização e estabelece o vínculo.

No UDP, nada disso existe:

- Não há `connect()` no cliente para iniciar uma sessão (embora o sistema operacional permita um `connect()` virtual apenas para fixar o endereço de destino no *socket*, nenhuma mensagem é enviada à rede).
- Não há `accept()` no servidor. O servidor não 'aceita' clientes; ele apenas aceita pacotes

Paradigma da caixa de correio - o servidor UDP pode ser imaginado como uma caixa de correio residencial - o servidor apenas escuta:

- O servidor executa a função `bind()` em uma porta específica (exemplo: porta 8080) - isso equivale a colocar um número na caixa de correio.
- Em seguida, ele entra em um *loop* infinito chamando uma função como `recvfrom()`.
- O servidor fica parado ali - ele não sabe quem vai enviar uma mensagem, quando vai enviar, ou quantas pessoas vão enviar ao mesmo tempo.

Especificar o endereço a cada pacote:

- No TCP, após a conexão ser aceita, você usa as funções `send()` e `recv()`. Você não precisa dizer para quem está enviando a mensagem, pois o *socket* já está 'amarrado' àquele cliente.
- No UDP, como não há conexão, cada pacote (chamado de datagrama) é uma entidade 100% independente. Por isso, as funções utilizadas são `sendto()` e `recvfrom()`:
 - Quem envia (`sendto`): Precisa anexar explicitamente o IP e a porta de destino em cada mensagem disparada. Se o cliente quiser enviar 10 mensagens para o mesmo servidor, ele terá que passar o endereço do servidor 10 vezes.
 - Quem recebe (`recvfrom`): Quando o servidor recebe um pacote, a função preenche automaticamente uma estrutura de dados com o IP e a porta de quem enviou. Se o servidor quiser responder, ele precisa ler esse endereço gerado e usá-lo para enviar o pacote de volta.

Para fixar a diferença, pense na analogia clássica do 'Correio vs. Telefone':

- TCP é uma ligação telefônica (**orientado a conexão**): Você disca (*connect*), a outra pessoa atende (*accept*), a linha fica aberta. Enquanto vocês conversam, você não precisa discar o número a cada palavra dita. Se a linha cair, você percebe imediatamente.
- UDP é o envio de cartas (**não-orientado a conexão**): Você escreve uma mensagem, coloca em um envelope e escreve o endereço de destino. Se quiser enviar outra mensagem, precisa pegar outro envelope e escrever o endereço todo de novo. Você joga a carta na caixa de correio e não tem nenhuma garantia de que ela vai chegar, nem sabe se o destinatário mudou de endereço. Se ele quiser te responder, ele terá que olhar o endereço de remetente que você escreveu no verso.

Essa crueza do UDP faz com que o programador entenda que o controle de fluxo, a retransmissão de pacotes perdidos e a ordenação de mensagens (tudo o que o TCP faz automaticamente) são, na verdade, *softwares* complexos rodando por trás dos panos.

5 Chat

Nesta seção, exploraremos o desenvolvimento de um sistema de **chat**.

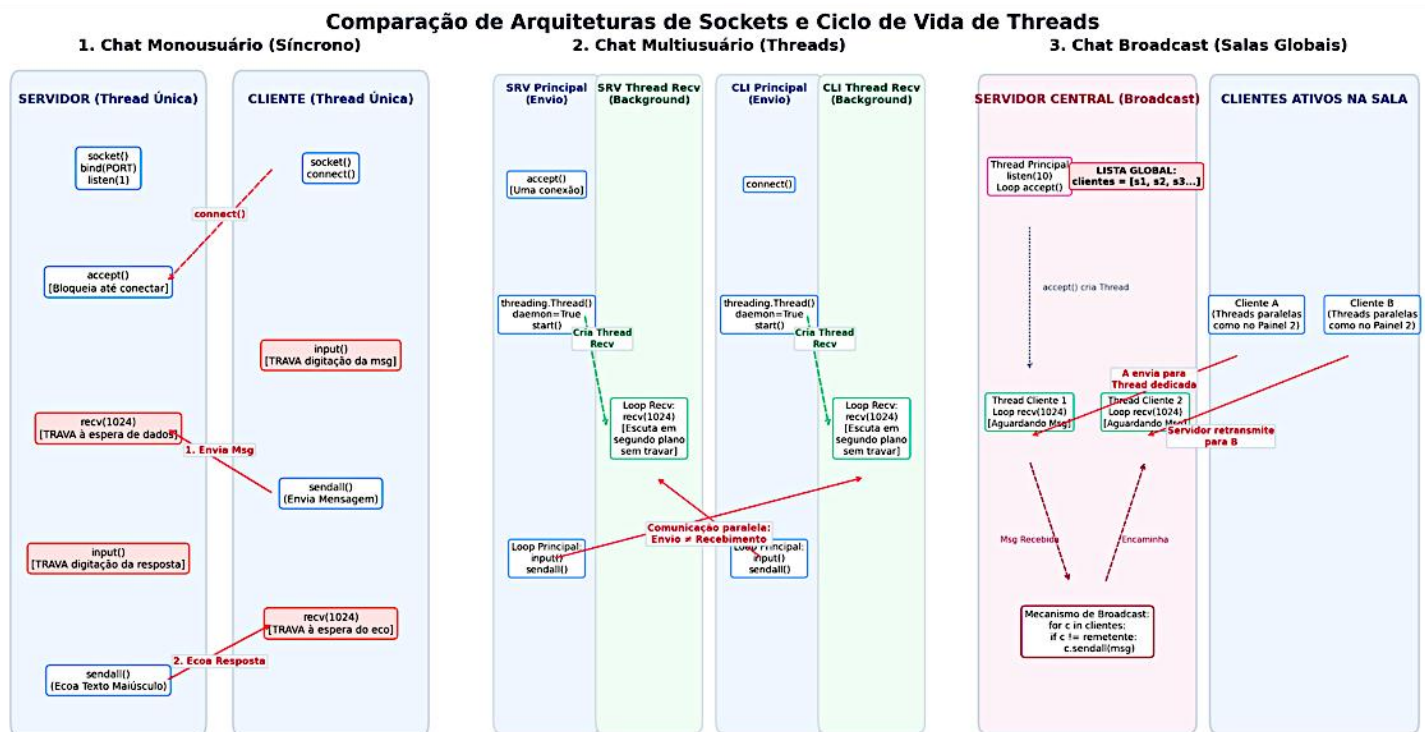
Serão desenvolvidas 3 versões do **chat**:

1. Cliente-servidor unicliente (*single-client server*) com 2 usuários do tipo síncrono/bloqueante (*ping-pong*)
2. Cliente-servidor (2 usuários) com *multi-thread* (não bloqueante)
3. Multiusuário (*multi-client server*) com *broadcast*

Primeiramente, analisaremos as limitações de um modelo **cliente-servidor (2 usuários) síncrono**, onde cliente e servidor são forçados a seguir um fluxo rígido de '*ping-pong*' por turnos, travando no recebimento de dados ou na entrada de dados do teclado.

Em seguida, esse problema será resolvido evoluindo para um modelo **cliente-servidor multi-thread**. Este modelo é implementado por meio de técnicas de programação concorrente usando **threads** do Python, permitindo que processos de recebimento rodem em segundo plano enquanto o fluxo principal da aplicação fica totalmente livre para enviar mensagens a qualquer momento.

Por fim, o capítulo é encerrado com uma versão de **broadcast** centralizado. O servidor centralizado pode gerenciar múltiplas conexões simultâneas mantendo uma lista global de clientes ativos. Quando qualquer participante envia uma mensagem, o servidor a recebe e a retransmite (faz o *broadcast*) para todos os outros conectados na sala global. Do lado do cliente, a execução de *threads* de recebimento em *background* garante que as mensagens dos outros usuários apareçam instantaneamente na tela, sem interromper o fluxo de digitação de quem está escrevendo no terminal.



5.1 Monousuário (2 Usuários)

O código da versão **monousuário** implementa um sistema de *chat* síncrono básico utilizando o protocolo TCP. Ele ilustra o funcionamento de *sockets* em uma comunicação direta, mas evidencia as limitações de um modelo de execução sequencial e bloqueante.

Servidor:

```
import socket
import sys

HOST = sys.argv[1] if len(sys.argv) > 1 else '0.0.0.0'
PORT = int(sys.argv[2]) if len(sys.argv) > 2 else 5000

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind((HOST, PORT))
server.listen(1)

print(f"[*] Servidor SÍNCRONO aguardando conexão em {HOST}:{PORT}...")
client_socket, client_address = server.accept()
print(f"[+] Cliente conectado: {client_address}")

try:
    while True:
        # PROBLEMA 1: O servidor trava aqui e fica esperando o cliente falar algo.
        # Ele é incapaz de enviar uma mensagem por conta própria antes de receber dados.
        data = client_socket.recv(1024)
        if not data:
            break
        print(f"\n[Cliente diz]: {data.decode('utf-8').strip()}")

        # PROBLEMA 2: Após receber, o servidor trava no input do teclado.
        # Se o cliente enviar outra mensagem enquanto o servidor digita, ela ficará presa no buffer.
        mensagem = input("[Sua resposta]: ")
        client_socket.sendall((mensagem + "\n").encode('utf-8'))

        if mensagem.lower() == 'sair':
            break
finally:
    client_socket.close()
    server.close()
```

Cliente:

```
import socket
import sys

HOST = sys.argv[1] if len(sys.argv) > 1 else '127.0.0.1'
PORT = int(sys.argv[2]) if len(sys.argv) > 2 else 5000

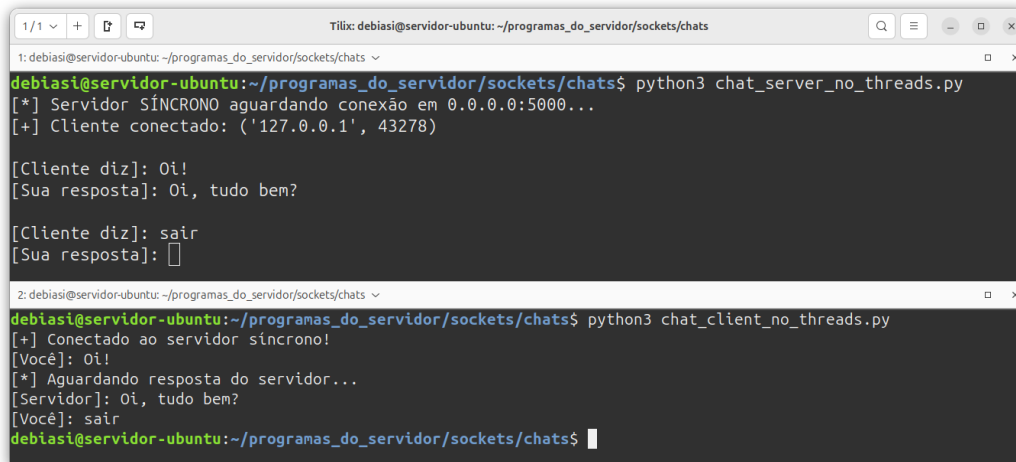
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))
print("[+] Conectado ao servidor síncrono!")

try:
    while True:
        # O cliente é forçado a seguir um protocolo rígido de "Ping-Pong" (Turnos)
        mensagem = input("[Você]: ")
        client.sendall((mensagem + "\n").encode('utf-8'))

        if mensagem.lower() == 'sair':
            break

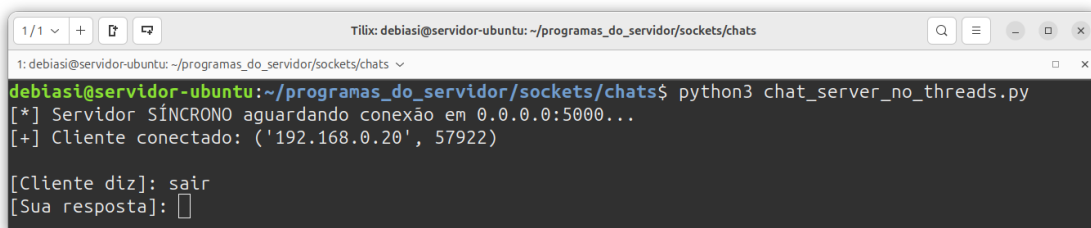
        print("[*] Aguardando resposta do servidor...")
        # O cliente trava aqui e não consegue digitar nada até o servidor responder
        data = client.recv(1024)
        if not data:
            break
        print(f"[Servidor]: {data.decode('utf-8').strip()}")
finally:
    client.close()
```

Demonstração (mesma máquina):



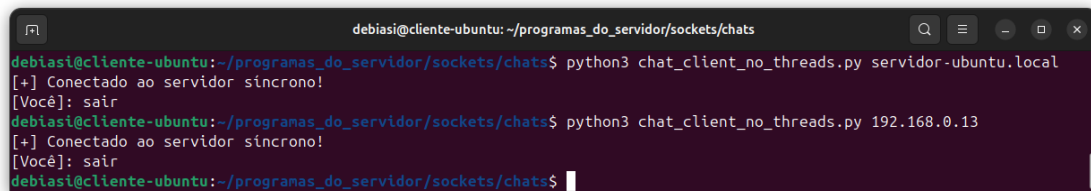
The image shows two terminal windows from a Tilix session on a server. The top window shows the execution of `python3 chat_server_no_threads.py`. It starts by displaying 'Servidor SÍNCRONO aguardando conexão em 0.0.0.0:5000...', then reports a client connection from '127.0.0.1' on port 43278. It then shows a conversation: the client says 'Oi!', the server responds 'Oi, tudo bem?', the client says 'sair', and the server responds with an empty prompt. The bottom window shows the execution of `python3 chat_client_no_threads.py`. It reports connecting to the server synchronously, shows the user input 'Oi!', the server's response 'Oi, tudo bem?', the user input 'sair', and the server's final empty prompt.

Demonstração (máquinas diferentes) – abaixo o servidor:



The image shows a terminal window from a Tilix session on a server. It shows the execution of `python3 chat_server_no_threads.py`. It starts by displaying 'Servidor SÍNCRONO aguardando conexão em 0.0.0.0:5000...', then reports a client connection from '192.168.0.20' on port 57922. It then shows the client sending 'sair' and the server responding with an empty prompt.

Cliente (o primeiro exemplo usa o *hostname* e o segundo usa o endereço IP):



The image shows a terminal window from a Tilix session on a client machine. It shows two separate executions of `python3 chat_client_no_threads.py`. The first execution uses the hostname 'servidor-ubuntu.local' and reports connecting to the server synchronously, followed by the user input 'sair'. The second execution uses the IP address '192.168.0.13' and also reports connecting to the server synchronously, followed by the user input 'sair'.

Breve explicação de como cada parte funciona e seus principais problemas:

- **Servidor monousuário:** O servidor inicializa um *socket* TCP, associa-se a uma porta (por padrão, a 5000), define o limite de uma conexão em espera e aguarda o cliente se conectar usando `accept()`. Uma vez conectado, ele entra em um *loop* de conversação que apresenta dois problemas críticos de bloqueio:
 - **Bloqueio de recebimento (problema 1):** O fluxo do servidor trava na linha `client_socket.recv(1024)` aguardando o cliente falar algo. Ele é totalmente incapaz de iniciar ou enviar uma mensagem por conta própria antes de receber algum dado.
 - **Bloqueio de teclado (problema 2):** Após receber a mensagem do cliente, o servidor trava novamente na função de leitura do teclado (`input("[Sua resposta]: ")`). Se o cliente tentar enviar novas mensagens enquanto o servidor estiver digitando, essas mensagens ficarão presas no buffer da rede e não aparecerão na tela até que o servidor termine de responder.
- **Cliente monousuário:** O cliente conecta-se ativamente ao IP e porta do servidor usando `client.connect()`. Ele é obrigado a seguir um protocolo rígido de turnos, conhecido como **'ping-pong'**:
 - O cliente primeiro solicita a digitação do usuário via `input("[Você]: ")` e envia o texto ao servidor.
 - Logo após enviar, ele entra imediatamente em estado de espera bloqueante em `client.recv(1024)`. Enquanto o servidor não enviar uma resposta de volta, o terminal do cliente fica congelado e o usuário não consegue digitar novas mensagens.

5.2 Multiusuário

A **versão multiusuário** resolve esses travamentos utilizando **threads** para separar a recepção de dados do fluxo de digitação, permitindo que ambas as tarefas ocorram de forma concorrente e paralela.

Servidor:

```
import socket
import sys
import threading

HOST = sys.argv[1] if len(sys.argv) > 1 else '0.0.0.0'
PORT = int(sys.argv[2]) if len(sys.argv) > 2 else 5000

# Função que roda em paralelo apenas para RECEBER mensagens do cliente
def receber_mensagens(client_socket, client_address):
    while True:
        try:
            data = client_socket.recv(1024)
            if not data:
                print(f"\n[*] O cliente {client_address[0]} desconectou.")
                break

            print(f"\n[Cliente {client_address[0]}]: {data.decode('utf-8').strip()}")
            print("[Você]: ", end="", flush=True)
        except:
            break

    client_socket.close()

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind((HOST, PORT))
server.listen(1)

print(f"\n[*] Sala de Chat aguardando conexão em {HOST}:{PORT}...")

try:
    # O accept() roda uma única vez para pegar o primeiro cliente que chegar
    client_socket, client_address = server.accept()

    # Passamos o 'client_address' (que contém o IP real) para a Thread usar no print
    print(f"[+] Cliente real conectado do IP: {client_address[0]} (Porta: {client_address[1]})")

    # Inicia a Thread para escutar esse cliente específico em background
    t = threading.Thread(target=receber_mensagens, args=(client_socket, client_address), daemon=True)
    t.start()

    # O loop principal do servidor fica livre apenas para ENVIAR mensagens
    while True:
        mensagem = input("[Você]: ")
        if mensagem.lower() == 'sair':
            break

        client_socket.sendall((mensagem + "\n").encode('utf-8'))

    client_socket.close()
except KeyboardInterrupt:
    print("\n[*] Chat encerrado pelo servidor...")
finally:
    server.close()
```

Cliente:

```
import socket
import sys
import threading

# 1. Configuração dos valores padrões (Localhost na porta 5000)
HOST = '127.0.0.1'
PORT = 5000

# 2. Tratamento dos argumentos da linha de comando
if len(sys.argv) > 1:
    HOST = sys.argv[1]

if len(sys.argv) > 2:
    try:
        PORT = int(sys.argv[2])
    except ValueError:
        print("Erro: A porta deve ser um número inteiro.")
        sys.exit(1)

# Função executada em background APENAS para escutar o servidor
def receber_mensagens(client_socket):
    while True:
        try:
            data = client_socket.recv(1024)
            if not data:
                print("\n[*] O servidor encerrou o chat.")
                break

            print(f"\n[Servidor]: {data.decode('utf-8').strip()}")
            print("[Você]: ", end="", flush=True)

        except (OSError, socket.error):
            # Captura silenciosamente o Bad file descriptor (Errno 9) ou Connection reset
            # quando o cliente ou o servidor fecham o socket abruptamente
            break
        except:
            break

    # Removemos o print daqui para evitar mensagens duplicadas ou fora de hora
    try:
        client_socket.close()
    except:
        pass

# 3. Inicialização do Socket TCP
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

try:
    print(f"\n[*] Conectando ao chat em {HOST}:{PORT}...")
    client.connect((HOST, PORT))
    print("[+] Conectado à sala de chat!")
    print("Digite suas mensagens livremente. Digite 'sair' para encerrar.")
    print("-" * 50)

    # 4. Dispara a Thread em background para RECEBER dados do servidor
    t = threading.Thread(target=receber_mensagens, args=(client,), daemon=True)
    t.start()

    # 5. O loop principal do script fica livre apenas para ENVIAR dados
    while True:
        mensagem = input("[Você]: ")
        mensagem_limpa = message_clean = mensagem.strip()

        if not mensagem_limpa:
            continue

        if mensagem_limpa.lower() == 'sair':
            try:
                client.sendall("sair\n".encode('utf-8'))
            except:
                pass
            break
```

```

try:
    client.sendall((mensagem_limpa + "\n").encode('utf-8'))
except (OSError, socket.error):
    print("\n[-] Erro: Não foi possível enviar a mensagem. O servidor caiu.")
    break
except KeyboardInterrupt:
    print("\n[*] Você saiu do chat (Ctrl+C).")
except ConnectionRefusedError:
    print(f"[-] Erro: Não foi possível conectar ao chat em {HOST}:{PORT}.")
except Exception as e:
    # Este bloco agora só pegará erros REAIS de inicialização, não de encerramento
    print(f"[-] Erro inesperado na conexão: {e}")
finally:
    # Fecha o socket de forma segura no final do script
    try:
        client.close()
    except:
        pass
print("[*] Programa finalizado.")

```

Explicações:

1. Separação de responsabilidades (paralelismo de redes):

- **Thread de recebimento (background):** Tanto no servidor quanto no cliente, uma *thread* secundária é disparada especificamente para rodar a função `receber_mensagens`. Essa *thread* fica em um *loop* infinito executando a leitura do *socket* (`client_socket.recv(1024)`). Como ela roda de forma assíncrona e em paralelo, o bloqueio do `recv` (esperando dados do outro lado) não afeta mais o fluxo principal do programa.
- **Fluxo principal (envio):** O fluxo principal da aplicação (a *thread* primária) fica totalmente livre e dedicada a capturar a entrada do usuário através do teclado (`input("[Você]: ")`) e enviar as mensagens (`sendall`).

2. Funcionamento prático no servidor multiusuário:

- O servidor aceita a conexão de um cliente via `accept()`.
- Imediatamente, ele inicializa uma *thread* dedicada (`threading.Thread`) passando a conexão do cliente e o seu endereço. A *thread* é disparada via `t.start()` e assume a escuta de mensagens em *background*.
- O *loop* principal do servidor segue em frente e só trava quando aguarda o próprio usuário digitar algo no terminal. Se o cliente enviar algo enquanto o servidor digita, a *thread* em *background* recebe os dados instantaneamente e os exibe no console.
- O argumento `daemon=True` define a *thread* criada como uma **thread daemon** (*thread* de 2º plano ou de serviço). Por padrão o Python aguarda que *todas* as *threads* em execução finalizem suas tarefas antes de fechar o programa por completo. Ao definir `daemon=True`, é sinalizado ao Python que essa *thread* é secundária e não deve impedir o fechamento do programa. Ao digitar 'sair' ou pressionar `Ctrl + C`, o *loop* principal termina e a *thread* principal é encerrada. Como a *thread* de recebimento de mensagens é do tipo *daemon*, o Python fecha o programa na hora, derrubando a *thread* secundária automaticamente. Sem isso, se o usuário digitasse "sair", a *thread* principal terminaria, mas **o terminal ficaria totalmente travado e congelado**. Isso aconteceria porque a *thread* secundária de recebimento continuaria ativa, bloqueada indefinidamente na chamada de rede `recv` esperando dados que talvez nunca cheguem, impedindo o programa de fechar.

3. Funcionamento prático no cliente multiusuário:

- Após estabelecer a conexão física com o servidor (*connect*), o cliente também dispara uma *thread* em *background* (`threading.Thread`) para rodar `receber_mensagens`.
- O *loop* principal do cliente fica responsável unicamente por receber o input do usuário e despachá-lo pela rede (`client.sendall`). Se o servidor enviar uma resposta a qualquer momento, a *thread* do cliente a exibe na tela sem interromper a digitação em andamento.

5.3 Broadcast

Esta versão (***broadcast***) eleva o sistema de comunicação para um modelo **multiusuário real e descentralizado**, permitindo que múltiplas pessoas conversem simultaneamente em uma ‘sala global’. Enquanto a versão multiusuário anterior limitava-se a conectar estritamente um único cliente a um único servidor, a arquitetura de *broadcast* foi desenhada para gerenciar um número indeterminado de conexões ao mesmo tempo.

Servidor:

```
import socket
import sys
import threading

HOST = sys.argv[1] if len(sys.argv) > 1 else '0.0.0.0'
PORT = int(sys.argv[2]) if len(sys.argv) > 2 else 5000

# Lista global para rastrear todos os clientes conectados
clientes = []

def gerenciar_cliente(client_socket, client_address):
    print(f"[+] {client_address} entrou no chat.")
    # Adiciona o novo cliente à lista
    clientes.append(client_socket)

    try:
        while True:
            data = client_socket.recv(1024)
            if not data:
                break

            mensagem = data.decode('utf-8').strip()
            if mensagem.lower() == 'sair':
                break

            formato_msg = f"\n[{client_address}]: {mensagem}\n[Você]: "
            print(f"Transmitindo de {client_address}: {mensagem}")

            # BROADCAST: Envia para todos os OUTROS clientes conectados
            for c in clientes:
                if c != client_socket:
                    try:
                        c.sendall(formato_msg.encode('utf-8'))
                    except:
                        # Remove clientes que caíram ou desconectaram abruptamente
                        if c in clientes:
                            clientes.remove(c)

    except Exception as e:
        print(f"[-] Erro com {client_address}: {e}")
    finally:
        if client_socket in clientes:
            clientes.remove(client_socket)
        client_socket.close()
        print(f"[-] {client_address} saiu do chat.")

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind((HOST, PORT))
server.listen(10) # Suporta até 10 conexões na fila
print(f"[*] Servidor de Broadcast Centralizado rodando em {HOST}:{PORT}")

try:
    while True:
        client_socket, client_address = server.accept()
        t = threading.Thread(target=gerenciar_cliente, args=(client_socket, client_address), daemon=True)
        t.start()
except KeyboardInterrupt:
    print("\n[*] Fechando servidor central...")
finally:
    server.close()
```

Cliente:

```
import socket
import sys
import threading

HOST = sys.argv[1] if len(sys.argv) > 1 else '127.0.0.1'
PORT = int(sys.argv[2]) if len(sys.argv) > 2 else 5000

def receber_mensagens(client_socket):
    while True:
        try:
            data = client_socket.recv(1024)
            if not data:
                print("\n[*] Você foi desconectado do servidor central.")
                break

            # O servidor já manda a mensagem formatada com o IP/Porta de quem enviou
            print(data.decode('utf-8'), end="", flush=True)
        except:
            break
    client_socket.close()

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

try:
    print(f"[*] Conectando ao Servidor de Broadcast em {HOST}:{PORT}...")
    client.connect((HOST, PORT))
    print("[+] Conectado à sala global! Todos os alunos verão suas mensagens.\n")
    print("Digite suas mensagens livremente. Digite 'sair' para encerrar.")
    print("-" * 50)

    # A Thread de recebimento garante que as mensagens dos outros alunos
    # apareçam na tela imediatamente, mesmo se este aluno estiver digitando
    t = threading.Thread(target=receber_mensagens, args=(client,), daemon=True)
    t.start()

    while True:
        mensagem = input("[Você]: ")
        mensagem_limpa = mensagem.strip()

        if not mensagem_limpa:
            continue

        if mensagem_limpa.lower() == 'sair':
            client.sendall("sair\n".encode('utf-8'))
            break

        client.sendall((mensagem_limpa + "\n").encode('utf-8'))
except KeyboardInterrupt:
    print("\n[*] Você saiu da sala (Ctrl+C).")
except ConnectionRefusedError:
    print(f"[-] Erro: O servidor de broadcast não está rodando em {HOST}:{PORT}.")
finally:
    client.close()
    print("[*] Programa finalizado.")
```

Explicações:

1. **Servidor de *broadcast* (concentrador de mensagens):** O servidor atua como um intermediário centralizador (*hub*). Ele não conversa diretamente com um usuário; em vez disso, ele escuta a todos e replica a mensagem de um para todos os demais. Seu funcionamento baseia-se nos seguintes elementos:
 - **Lista global de clientes:** O servidor mantém uma lista em memória chamada `clientes = []`. Ela é usada para registrar ativamente a referência de *socket* de todos os participantes que estão conectados no momento.
 - **Fila de conexões ampliada:** O comando `server.listen(10)` configura o *socket* do servidor para suportar uma fila de até 10 conexões pendentes simultâneas de novos clientes que estão tentando entrar no *chat*.

- **Loop de aceitação contínuo:** No bloco principal do programa, o servidor roda um *loop* infinito executando `server.accept()`. Sempre que um novo participante se conecta, o servidor cria uma *thread* dedicada para rodar a função `gerenciar_cliente` em segundo plano com `daemon=True`, iniciando-a imediatamente com `t.start()`. Feito isso, a *thread* principal do servidor volta instantaneamente para a linha do `accept()` para aguardar o próximo cliente.
- **O mecanismo de *broadcast*:** Dentro de `gerenciar_cliente`, o *socket* do novo usuário é adicionado à lista global `clientes.append(client_socket)`. A partir daí, a *thread* daquele cliente específico fica escutando a rede por mensagens. Quando uma mensagem chega, o servidor executa um loop varrendo a lista de conexões (`for c in clientes`) e retransmite a mensagem formatada para todos os outros *sockets*, **exceto** para o próprio cliente que a enviou (`if c != client_socket`).
- **Tratamento e limpeza de erros:** Se um cliente digitar "sair", fechar o terminal ou perder a conexão de forma abrupta, o bloco `finally` garante que o *socket* seja fechado e devidamente removido da lista global `clientes`. Isso evita que o servidor tente enviar dados para conexões inativas ou corrompidas.

2. **Cliente de *broadcast* (interface assíncrona):** O cliente de *broadcast* conecta-se ao servidor centralizador e precisa lidar com duas tarefas em paralelo: receber mensagens de outras pessoas a qualquer momento e enviar as suas próprias mensagens sem que uma tarefa trave a outra.

- **Thread de recebimento dedicada:** Após realizar o `connect()`, o cliente cria e inicia uma *thread* em segundo plano executando a função `receber_mensagens`. Ela fica em um *loop* infinito aguardando a chegada de bytes através de `client.recv(1024)`.
- **Formatação inteligente na tela:** Quando outros usuários falam, o servidor de *broadcast* envia o texto pré-formatado contendo a identificação do remetente e reposicionando o *prompt* de escrita do terminal (por exemplo, `\n[IP:Porta]: mensagem\n[Você]:`). O cliente simplesmente exibe o texto recebido de forma direta usando `print(data.decode('utf-8'), end="", flush=True)`. Isso impede que a mensagem recebida "atrole" ou quebre a linha em que o usuário está digitando no console.
- **Loop principal de envio:** O fluxo de execução primário do cliente (a *thread* principal) fica exclusivamente encarregado de ler o que o usuário digita via `input("[Você]: ")` e enviar o texto limpo para o servidor central por meio do método `client.sendall()`.

Exemplo de sessão:

```

1: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/chats
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/chats$ python3 chat_server_broadcast.py
[*] Servidor de Broadcast Centralizado rodando em 0.0.0.0:5000
[+] ('127.0.0.1', 59628) entrou no chat.
[+] ('127.0.0.1', 59632) entrou no chat.
[+] ('127.0.0.1', 49030) entrou no chat.
Transmitindo de ('127.0.0.1', 59628): Olá, sou o fulano!
Transmitindo de ('127.0.0.1', 59632): Oi oi, sou o beltrano!
Transmitindo de ('127.0.0.1', 49030): Prazer, sou o sicrano!

2: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/chats
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/chats$ python3 chat_client_broadcast.py
[*] Conectando ao Servidor de Broadcast em 127.0.0.1:5000...
[*] Conectado à sala global! Todos os alunos verão suas mensagens.
Digite suas mensagens livremente. Digite 'sair' para encerrar.
[Você]: Olá, sou o fulano!
[Você]:
[('127.0.0.1', 59632)]: Oi oi, sou o beltrano!
[Você]:
[('127.0.0.1', 49030)]: Prazer, sou o sicrano!
[Você]:

3: debiasi@servidor-ubuntu: ~/programas_do_servidor/sockets/chats
debiasi@servidor-ubuntu:~/programas_do_servidor/sockets/chats$ python3 chat_client_broadcast.py
[*] Conectando ao Servidor de Broadcast em 127.0.0.1:5000...
[*] Conectado à sala global! Todos os alunos verão suas mensagens.
Digite suas mensagens livremente. Digite 'sair' para encerrar.
[Você]:
[('127.0.0.1', 59628)]: Olá, sou o fulano!
[Você]:
[('127.0.0.1', 59632)]: Oi oi, sou o beltrano!
[Você]: Prazer, sou o sicrano!
[Você]:

```

Como exercício agora crie um sistema cliente/servidor de *chat* com as seguintes características:

- Seja multiusuário (até 10 conexões) e o servidor também poderá conversar
- Permita que os clientes se identifiquem com um apelido (*nickname*)
- Mostre o apelido juntamente com o endereço IP no *prompt* da conversa
- O comando 'ajuda' mostrará a imagem abaixo
- O comando 'usuarios' mostra uma relação dos usuários
- Ao digitar 'sair' no cliente, ele deverá ser removido do *chat*
- 'sair' no servidor fará ele sair do *chat*, mas ele deverá continuar recebendo e encaminhando as mensagens
- O comando 'entrar' no servidor fará com que ele entre novamente no *chat*
- O comando 'kick <nickname>' deverá expulsar o cliente do *chat*
- Ao digitar 'encerrar' no servidor, o *chat* é finalizado e os clientes fechados

Servidor:

Comandos disponíveis:

```
D:\programas\ad_2026\ProjetosPython\.venv\Scripts\python.exe D:\progrn
[*] Sala de Chat Multiusuário aguardando conexões em 0.0.0.0:5000...

=====
          COMANDOS DISPONÍVEIS NO SERVIDOR
=====
ajuda      - Mostra esta lista de comandos
sair       - Sai do chat (modo gerenciamento oculto)
entrar     - Volta a participar ativamente do chat
usuarios   - Lista todos os clientes conectados
kick <nick> - Desconecta um usuário pelo nickname
encerrar   - Derruba todos os clientes e fecha o servidor
=====
```

```
[Servidor]: [Servidor]: [+] fulano (192.168.0.13:39020) entrou no chat.
[Servidor]: [+] beltrano (192.168.0.20:49042) entrou no chat.
[Servidor]: [beltrano - 192.168.0.20]: oi
[Servidor]: [fulano - 192.168.0.13]: olá
[Servidor]: buuu!
[Servidor]: usuarios

--- 2 Usuário(s) Conectado(s) ---
- beltrano (192.168.0.20:49042)
- fulano (192.168.0.13:39020)
-----
[Servidor]: kick fulano
[*] Expulsando o usuário fulano...
[Servidor]: sair
[*] Você saiu do chat. Modo gerenciamento ativado.
[Painel-Srv]: entrar
[*] Você voltou para o chat!
[Servidor]: encerrar
[*] Avisando clientes e derrubando conexões...
[*] Servidor desligado com sucesso.
```

Cliente (fulano):

➤ python3 chat_client_multiuser.py 192.168.0.100

```
debiassi@servidor-ubuntu: ~/programas/ad_2026/sockets/
(.venv) debiasi@servidor-ubuntu:~/programas/ad_2026/sockets/
Escolha seu nome de usuário (Nickname): fulano
[*] Conectando ao chat em 192.168.0.100:5000...
[+] Conectado à sala de chat!
Digite 'sair' para encerrar.
-----
[Você]:
[+] beltrano (192.168.0.20:49042) entrou no chat.

[Você]:
[beltrano - 192.168.0.20]: oi

[Você]: olá
[Você]:
[Servidor]: buuu!

[Você]:
[SISTEMA]: Você foi expulso do servidor pelo Administrador.

[Você]:
[*] O servidor encerrou o chat ou você foi desconectado.
[*] Programa finalizado.
(.venv) debiasi@servidor-ubuntu:~/programas/ad_2026/sockets/
```

Cliente (beltrano):

```
debiassi@cliente-ubuntu: ~/programas/ad_2026/sockets/
(.venv) debiasi@cliente-ubuntu:~/programas/ad_2026/sockets/
Escolha seu nome de usuário (Nickname): beltrano
[*] Conectando ao chat em 192.168.0.100:5000...
[+] Conectado à sala de chat!
Digite 'sair' para encerrar.
-----
[Você]: oi
[Você]:
[fulano - 192.168.0.13]: olá

[Você]:
[Servidor]: buuu!

[Você]:
[*] fulano foi expulso do chat pelo Administrador.

[Você]:
[SISTEMA]: O Administrador do Servidor saiu do chat.

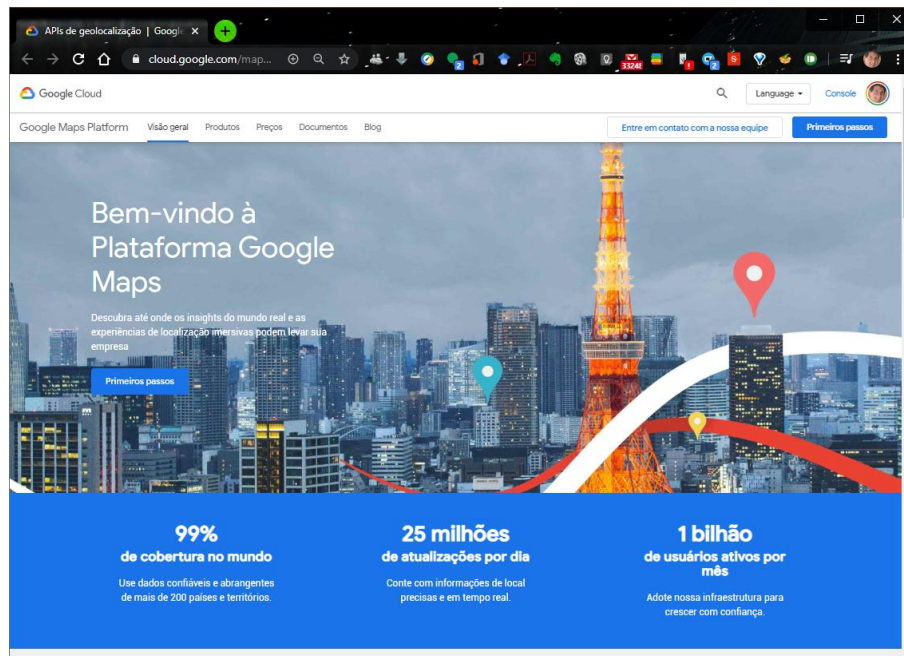
[Você]:
[SISTEMA]: O Administrador do Servidor entrou no chat.

[Você]:
[SISTEMA]: O Servidor está sendo encerrado imediatamente!

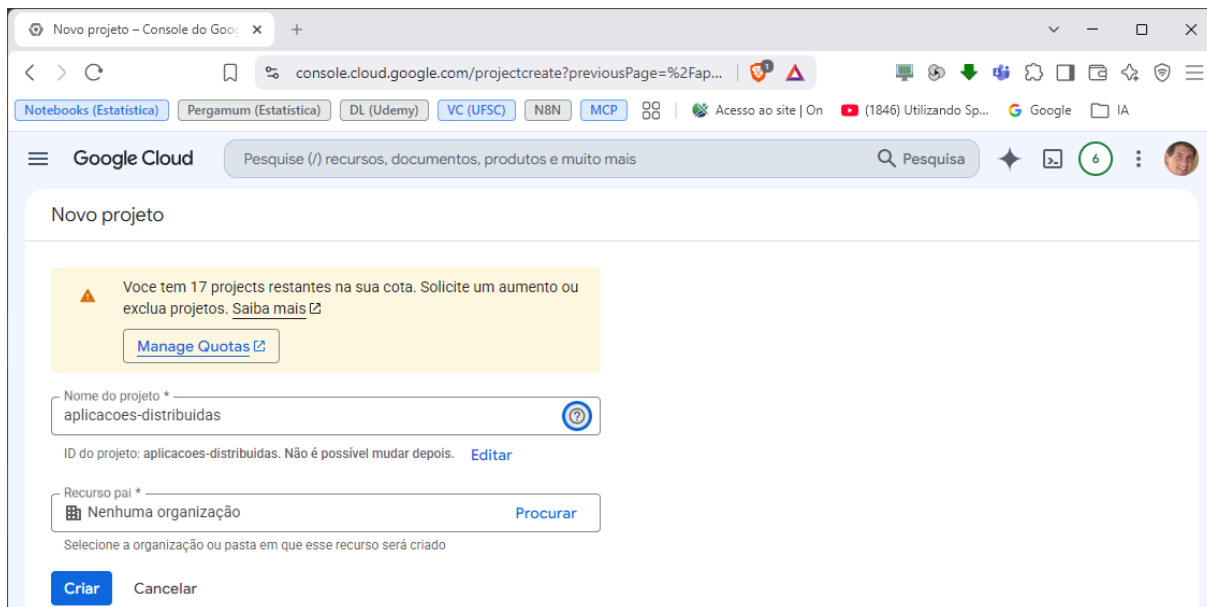
[Você]:
[*] O servidor encerrou o chat ou você foi desconectado.
[*] Programa finalizado.
(.venv) debiasi@cliente-ubuntu:~/programas/ad_2026/sockets/
```

6 APIs do Google Maps

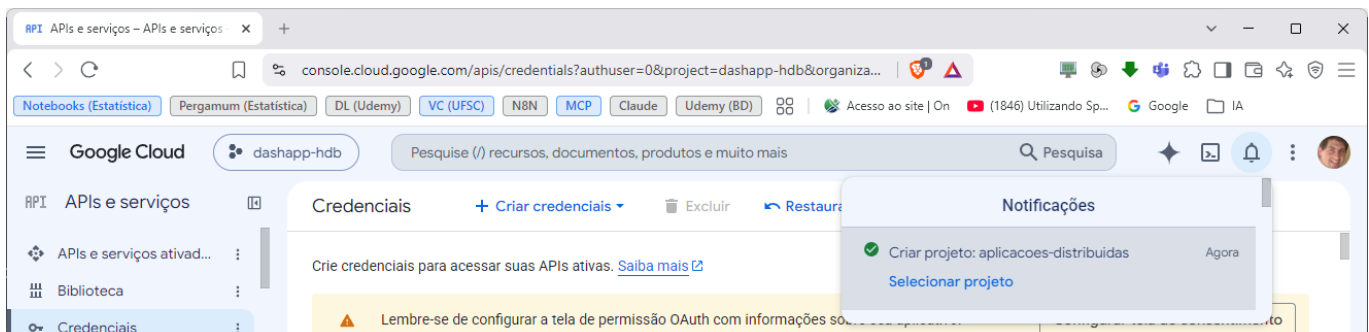
A [API do Google Maps](#) é **paga**, porém, permite que utilizemos alguns dos seus recursos gratuitamente. Caso o objetivo seja utilizar esta API em uma aplicação real, deve-se verificar os preços e planos, pois a opção gratuita é bem limitada.



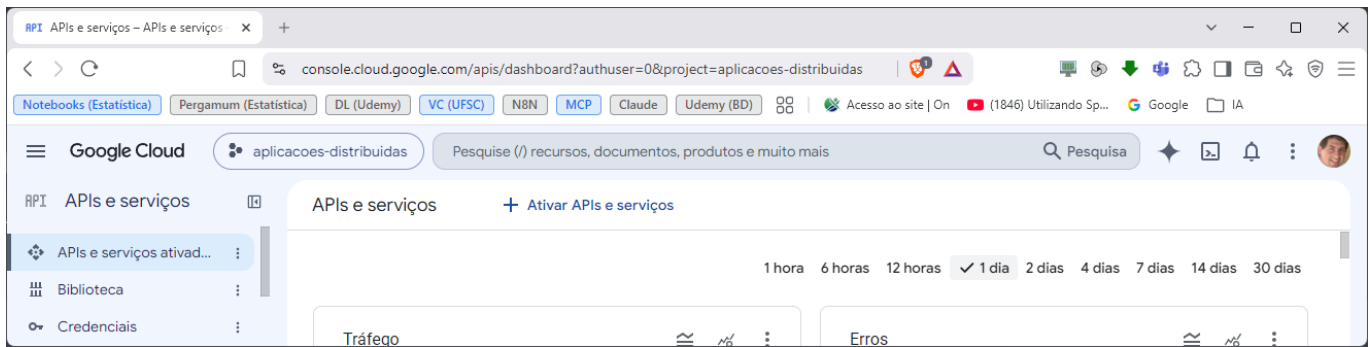
Crie o projeto chamado *'aplicacoes-distribuidas'*:



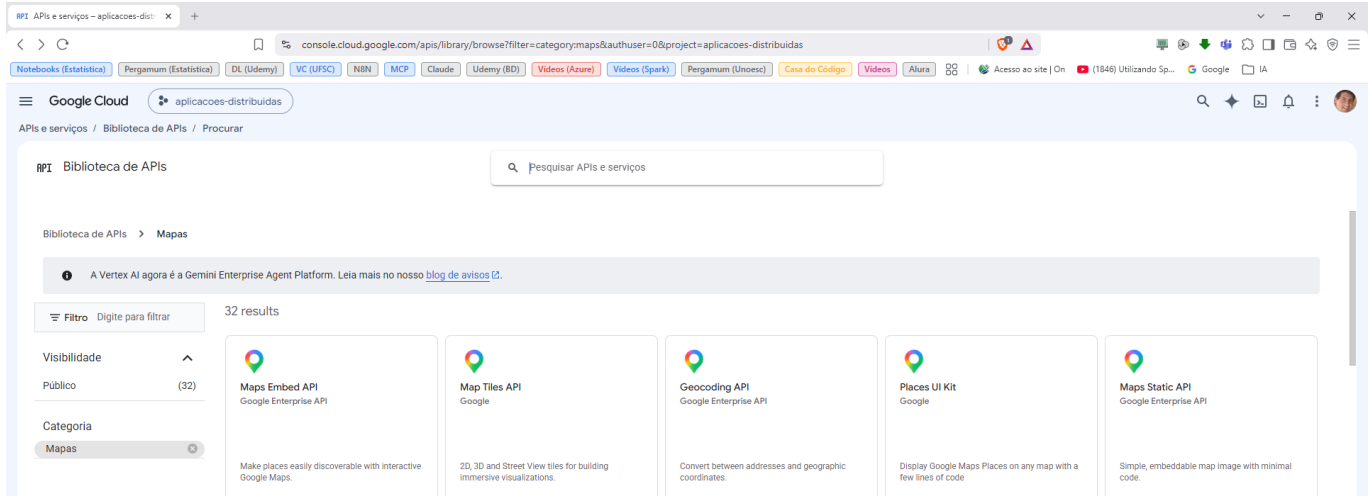
Ao final, selecione o projeto:



Vá em ‘APIs e serviços ativados’ e então clique no botão ‘+ Ativar APIs e serviços’.



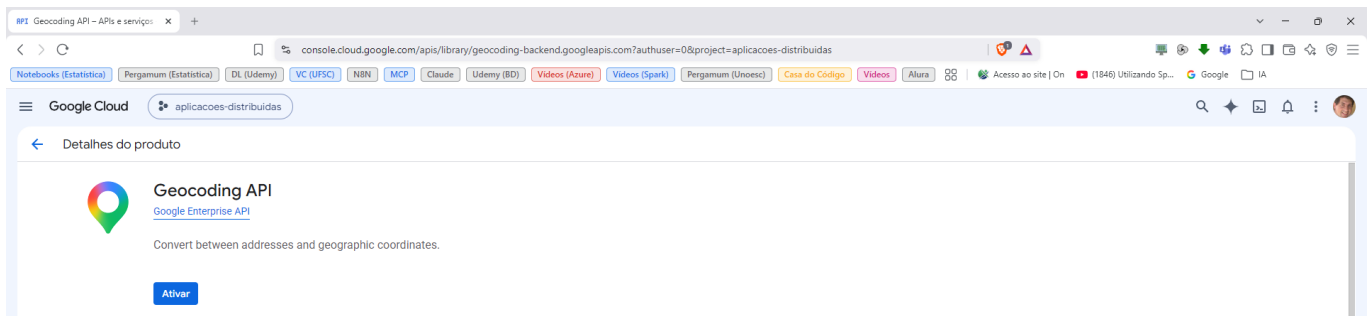
Escolha a categoria *Mapas* do lado esquerdo:



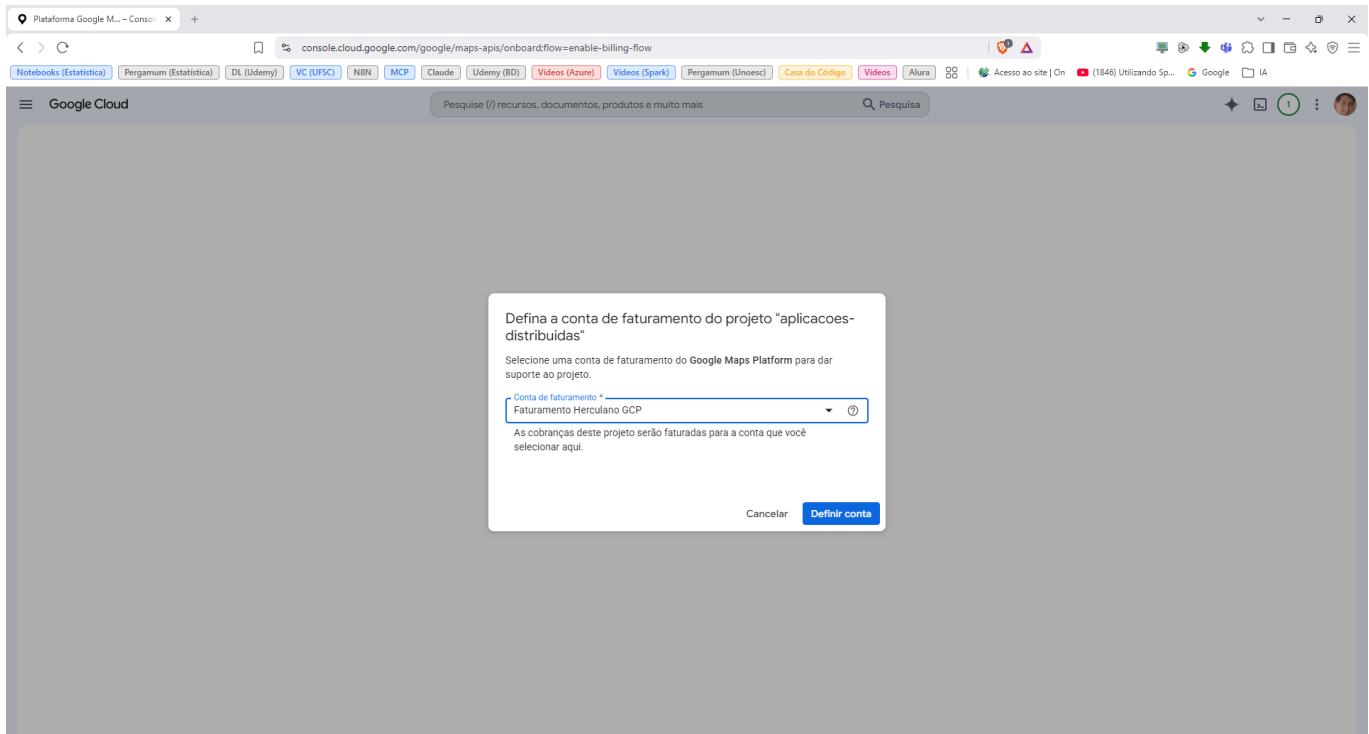
Ative as APIs abaixo:

- Geocoding API
- Geolocation API
- Map Tiles API
- Maps Embed API
- Maps JavaScript API
- Places API
- Places API (New)

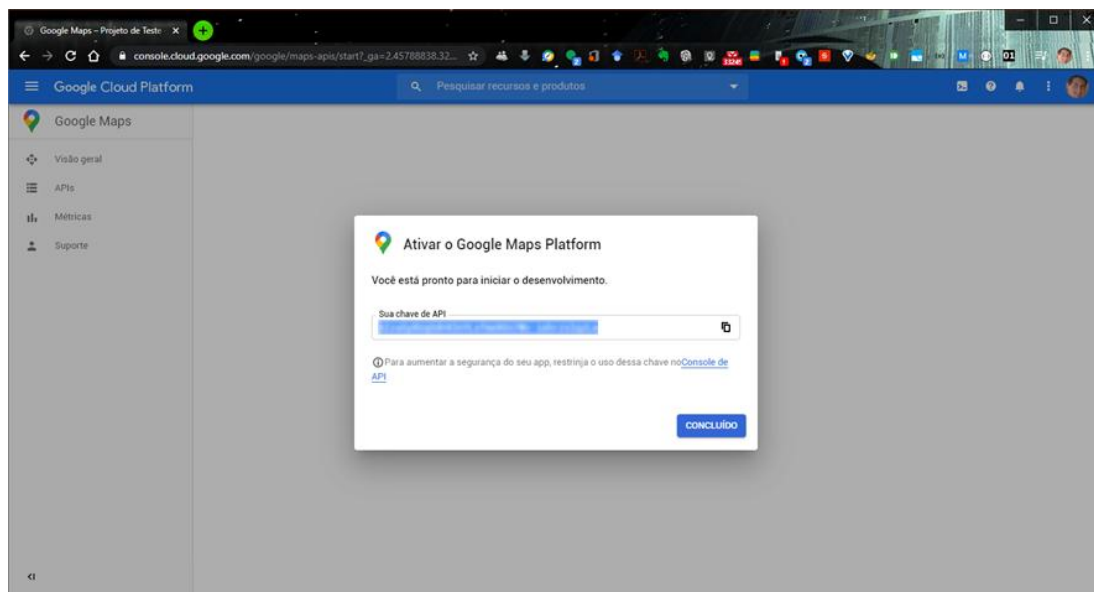
Exemplo:



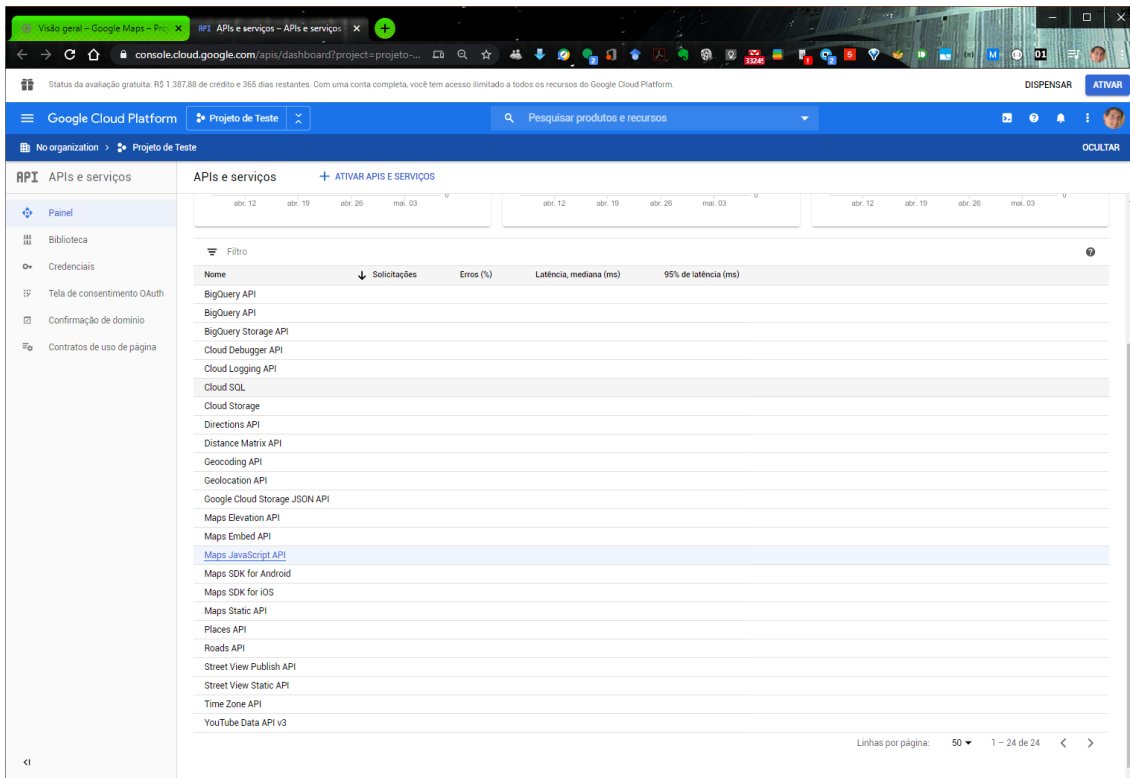
Ao ativar a primeira API será mostrada a tela para ‘Primeiros passos’ para criar/definir uma *conta de faturamento*.



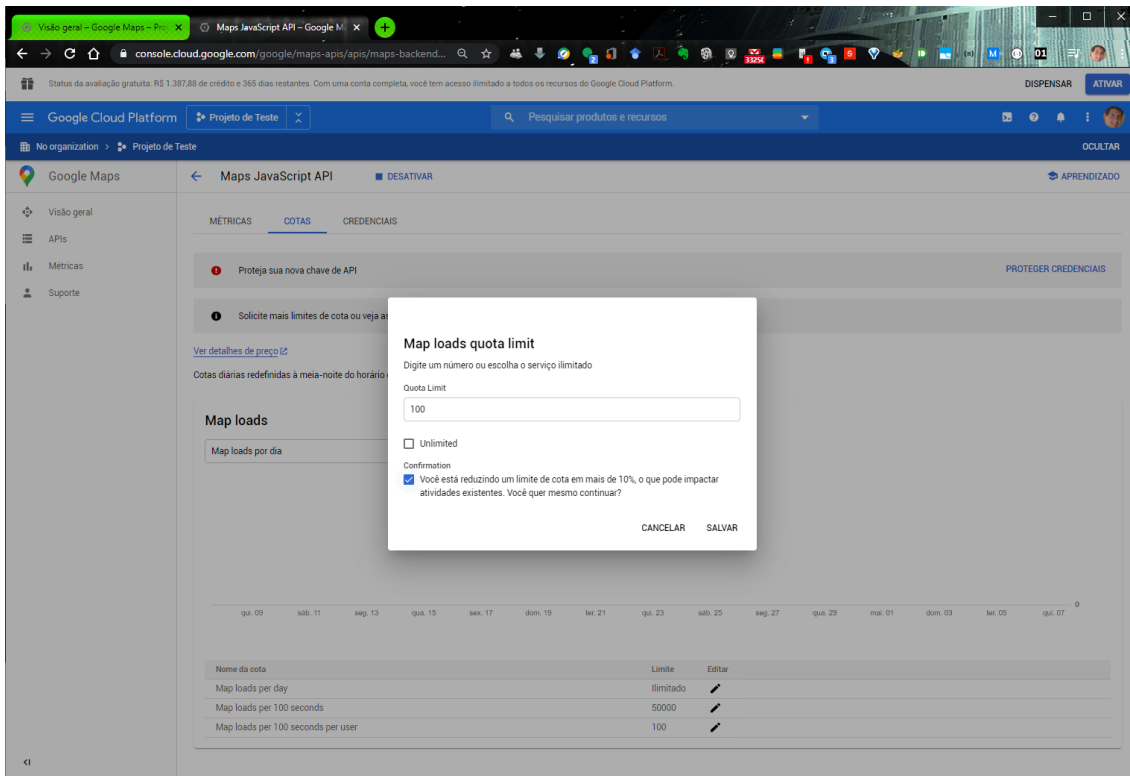
Geração da chave similar à esta 'AlzaSyA03zKo_Ey2LXS6y9SS12t9Fq4ZIA29lOk'.



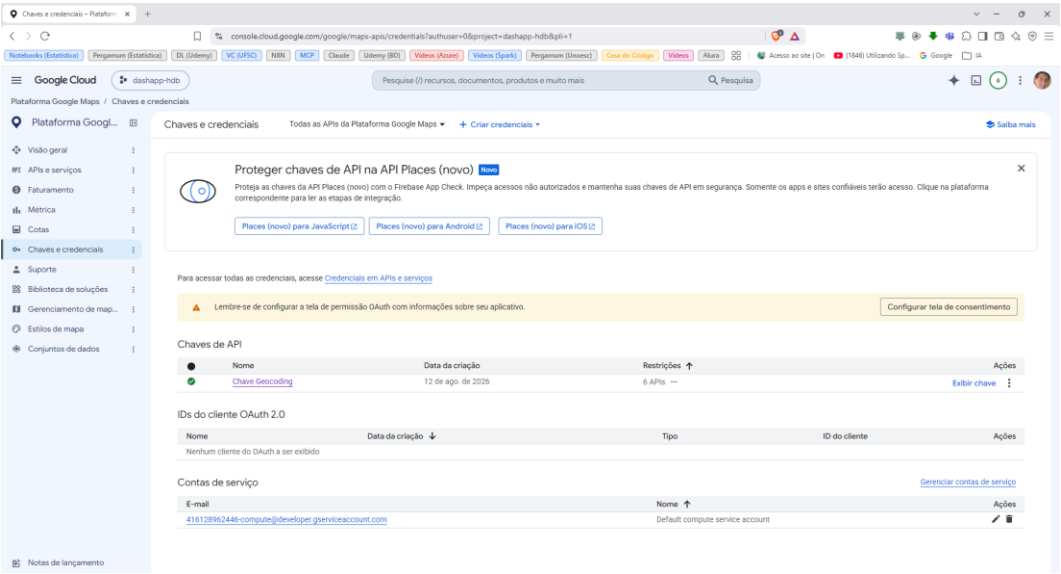
Abra o [dashboard](#) para configurar a limitação do número de visualizações e abra ‘*Maps JavaScript API*’.



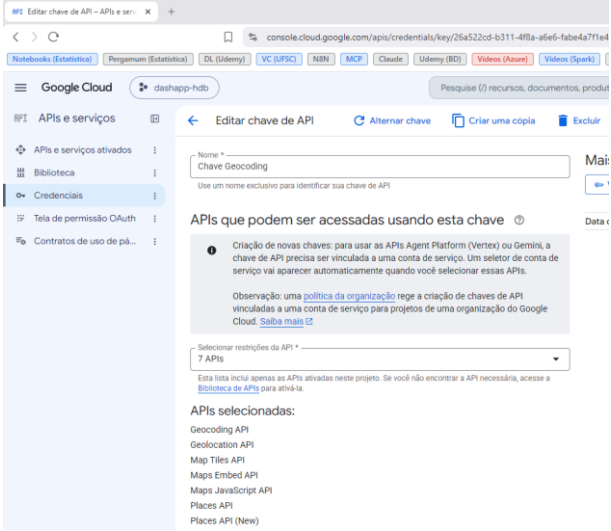
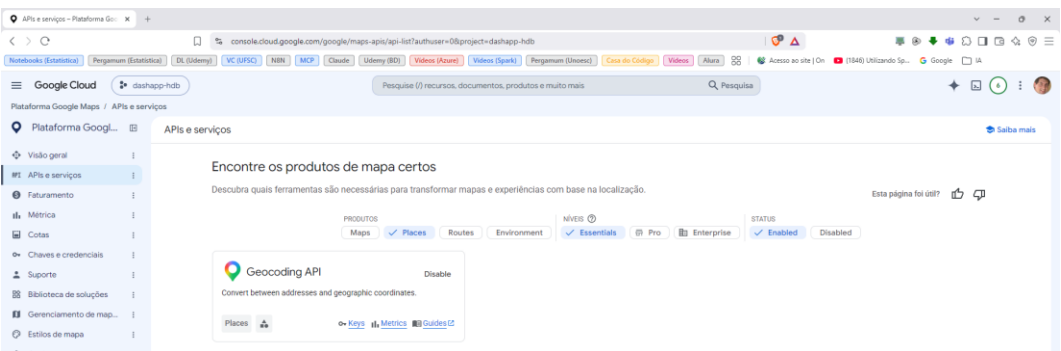
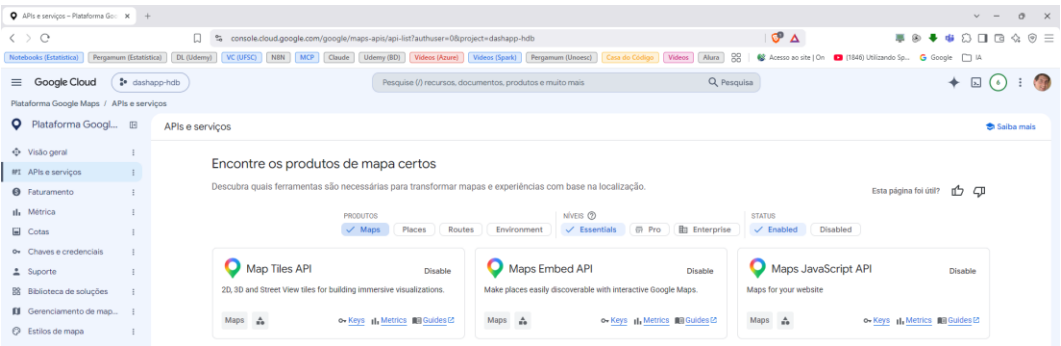
Escolha a aba cotas e modifique como abaixo:



Chaves e credenciais:



APIs habilitadas:



7 Estudo de Caso com Geolocalização

Entre na pasta base que contém o ambiente criado no início do roteiro e execute o comando abaixo:

➤ `source .venv/bin/activate`

Versão 1: Biblioteca *geopy*

- O código usa o serviço de geolocalização Nominatim, que é um motor de busca baseado nos dados do [OpenStreetMap](#) (OSM)
- Ele é totalmente gratuito e não exige chaves de acesso (API keys).
- O Nominatim possui uma política de uso que limita as requisições a **no máximo 1 por segundo**
- Ele não é recomendado para aplicações comerciais de grande escala ou buscas em lote (*bulk*) muito rápidas



Código:

```
from geopy.geocoders import Nominatim
from geopy.exc import GeocoderUnavailable, GeocoderServiceError

endereco = "Rua Paese, 198, Videira, SC, Brazil"

# user_agent deve ser único para identificar seu aplicativo
locator = Nominatim(user_agent="meu_agente_v1 (seuemail@provedor.com)")

try:
    print(f"Buscando endereço: {endereco}")

    # Realiza a geocodificação
    local = locator.geocode(endereco)

    # Verifica se o resultado existe (pode ser None se não encontrar)
    if local:
        print(f"Endereço Formatado: {local.address}")
        print(f"Latitude = {local.latitude}")
        print(f"Longitude = {local.longitude}")
    else:
        print("Erro: Nenhum resultado encontrado para este endereço.")

except GeocoderUnavailable:
    print("Erro: O serviço de geocodificação está indisponível no momento.")
except GeocoderServiceError as e:
    print(f"Erro no serviço: {e}")
except Exception as e:
    print(f"Erro inesperado: {e}")
```

Resultado:

```
Run v1 x
D:\programas\ad_2026\ProjetosPython\.venv\Scripts\python.exe D:\programas\ad_2026\ProjetosPython\geo\v1.py
Buscando endereço: Rua Paese, 198, Videira, SC, Brazil
Endereço Formatado: UNOESC Videira, 198, Rua Paese, Universitário, Videira, Santa Catarina, Região Sul, 89560-000, Brasil
Latitude = -26.9930379
Longitude = -51.1765569
Process finished with exit code 0
```

Versão 2a: Requisição à [API de geocodificação \(geocoding\)](#) da Google

- Módulo de requisições (`requests`) de alto nível
- Utilização de uma chave de autenticação para utilização do *webservice*
- Boas práticas de segurança (chave de API não está *hardcoded* no código-fonte, o que a deixaria totalmente exposta e sim está armazenada em arquivo externo `.env`)
- Se você enviar esse código para o GitHub, GitLab ou compartilhar com alguém, robôs vão capturar sua chave em poucos minutos – outras pessoas podem usar sua chave, gerando cobranças financeiras imensas no seu cartão de crédito cadastrado no Google Cloud
- Utilização de biblioteca para leitura de arquivos de configuração
- Tratamento de erros de rede/infraestrutura através de tratamento de exceções (`try...except`)
- Tratamento de erros relacionados às regras de negócio, como uma chave inválida ou endereço não encontrado (tratados no `if/elif`)
- Dados em formato JSON

Código:

```
import os
import requests
from dotenv import load_dotenv
from requests.exceptions import RequestException

# SEGURANÇA: Carrega a chave de API a partir do arquivo externo .env
load_dotenv()
API_KEY = os.getenv("GOOGLE_API_KEY")

# Endereço para busca e endpoint oficial da API do Google Maps
endereco = "Rua Paese, 198, Videira, SC, Brazil"
BASE_URL = 'https://maps.googleapis.com/maps/api/geocode/json'

parameters = {
    'address': endereco,
    'key': API_KEY
}

try:
    print(f"Buscando endereço: {endereco}")
    print(f"Enviando requisição HTTP GET para: {BASE_URL}")

    # ROBUSTEZ: 'timeout' de 10 segundos evita que o script trave se a internet cair
    response = requests.get(BASE_URL, params=parameters, timeout=10)

    # BOAS PRÁTICAS: Verifica erros de protocolo HTTP (ex: erro 403, 500) antes de ler o JSON
    response.raise_for_status()

    # Converte o corpo da resposta HTTP em um dicionário Python
    answer = response.json()

    print(f"\nStatus retornado no JSON da API: {answer.get('status')}")

    # TRATAMENTO DE REGRAS DE NEGÓCIO DA API
    if answer.get('status') == 'OK' and answer.get('results'):
        # Captura o primeiro resultado válido retornado pela busca
        primeiro_resultado = answer['results'][0]
        location = primeiro_resultado['geometry']['location']

        print(f"Endereço Formatado Oficial: {primeiro_resultado.get('formatted_address')}")
        print(f"Lat: {location['lat']}, Lng: {location['lng']}")
    elif answer.get('status') == 'REQUEST_DENIED':
        print("\n[Erro de Autenticação] A chave de API é inválida ou o faturamento não está ativo no Google Cloud.")
        print(f"Mensagem do Google: {answer.get('error_message')}")
    else:
        print(f"\n[Erro na Busca] O Google retornou o status: {answer.get('status')}")
        print("Verifique se o endereço está correto.")

# TRATAMENTO DE EXCEÇÕES DE REDE
except RequestException as e:
    print(f"\n[Erro de Conexão] Falha ao se comunicar com o servidor do Google: {e}")
except Exception as e:
    print(f"\n[Erro Inesperado] Ocorreu uma falha interna no script: {e}")
```

Versão 2b: Requisição à API de geocodificação (geocoding) da Google com HTTPX (síncrono)

- Utilização da biblioteca `httpx` para chamadas síncronas, uma alternativa moderna ao módulo `requests`
- Gerenciamento de conexões: Uso do gerenciador de contexto (`with httpx.Client()`) para garantir a abertura e o fechamento correto do cliente de rede, evitando vazamento de recursos
- Controle de indisponibilidade: Configuração de *timeout* de 10 segundos diretamente no cliente para evitar travamentos indefinidos caso a rede falhe.
- Separação explícita entre erros de protocolo (como HTTP 403 ou 500 capturados pelo `raise_for_status()`) e falhas físicas de conectividade/infraestrutura de rede através das exceções nativas do `httpx`.
- Validação das regras de negócio: Tratamento condicional (`if / elif / else`) baseado no *status* interno da resposta JSON enviada pelo Google Maps, isolando falhas de autenticação de problemas de busca de endereço.
- Manipulação de dados: Decodificação nativa do corpo da resposta para formato de dicionário Python (JSON).

Código:

```
import os
import httpx
from dotenv import load_dotenv

load_dotenv()
API_KEY = os.getenv("GOOGLE_API_KEY")

endereco = "Rua Paese, 198, Videira, SC, Brazil"
BASE_URL = 'https://maps.googleapis.com/maps/api/geocode/json'

parameters = {
    'address': endereco,
    'key': API_KEY
}

try:
    print(f"Buscando endereço (Síncrono): {endereco}")
    print(f"Enviando requisição HTTP GET para: {BASE_URL}")

    # Uso do gerenciador de contexto 'with' garante o fechamento correto das conexões
    with httpx.Client(timeout=10.0) as client:
        response = client.get(BASE_URL, params=parameters)
        response.raise_for_status()

    # Converte o corpo da resposta HTTP em um dicionário Python
    answer = response.json()

    print(f"\nStatus retornado no JSON da API: {answer.get('status')}")

    # TRATAMENTO DE REGRAS DE NEGÓCIO DA API
    if answer.get('status') == 'OK' and answer.get('results'):
        primeiro_resultado = answer['results'][0]
        location = primeiro_resultado['geometry']['location']

        print(f"Endereço Formatado Oficial: {primeiro_resultado.get('formatted_address')}")
        print(f"Lat: {location['lat']}, Lng: {location['lng']}")
    elif answer.get('status') == 'REQUEST_DENIED':
        print("\n[Erro de Autenticação] A chave de API é inválida ou o faturamento não está ativo no Google Cloud.")
        print(f"Mensagem do Google: {answer.get('error_message')}")
    else:
        print(f"\n[Erro na Busca] O Google retornou o status: {answer.get('status')}")
        print("Verifique se o endereço está correto.")

# TRATAMENTO DE EXCEÇÕES DE REDE ESPECÍFICAS DO HTTPX
except httpx.HTTPStatusError as e:
    print(f"\n[Erro de Protocolo] Resposta de erro do servidor (HTTP {e.response.status_code}): {e}")
except httpx.RequestError as e:
    print(f"\n[Erro de Conexão] Falha de rede ao se comunicar com o Google: {e}")
except Exception as e:
    print(f"\n[Erro Inesperado] Ocorreu uma falha interna no script: {e}")
```

Versão 2c: Requisição à API de geocodificação (geocoding) da Google com HTTPX (assíncrono)

- Programação assíncrona nativa: Uso da biblioteca `asyncio` e das palavras-chave `async/await` para executar operações de entrada/saída (I/O) sem bloquear a execução do programa
- Cliente assíncrono moderno: Utilização do `httpx.AsyncClient` via gerenciador de contexto assíncrono (`async with`) para gerenciar o ciclo de vida das conexões sem travar o fluxo principal
- Aplicação do operador `await` na chamada de rede, permitindo que o interpretador Python execute outras tarefas concorrentes enquanto aguarda a resposta do servidor do Google
- Tratamento granular de erros: Captura especializada de exceções de protocolo (`HTTPStatusError`) causadas por falhas de servidor e erros de conexão física (`RequestError`) específicos do ecossistema HTTPX
- Ponto de entrada assíncrono: Inicialização segura do *loop* de eventos do Python por meio do método `asyncio.run()`, permitindo o disparo correto da função assíncrona principal

Código:

```
import os
import asyncio
import httpx
from dotenv import load_dotenv

load_dotenv()
API_KEY = os.getenv("GOOGLE_API_KEY")

async def buscar_coordenadas():
    endereco = "Rua Paese, 198, Videira, SC, Brazil"
    BASE_URL = 'https://maps.googleapis.com/maps/api/geocode/json'

    parameters = {
        'address': endereco,
        'key': API_KEY
    }

    try:
        print(f"Buscando endereço (Assíncrono): {endereco}")
        print(f"Enviando requisição HTTP GET para: {BASE_URL}")

        # Uso do cliente assíncrono do httpx
        async with httpx.AsyncClient(timeout=10.0) as client:
            # A palavra 'await' pausa temporariamente ESTA função liberando o processador para outras tarefas
            response = await client.get(BASE_URL, params=parameters)
            response.raise_for_status()

        answer = response.json()
        print(f"\nStatus retornado no JSON da API: {answer.get('status')}")

        # TRATAMENTO DE REGRAS DE NEGÓCIO DA API
        if answer.get('status') == 'OK' and answer.get('results'):
            primeiro_resultado = answer['results'][0]
            location = primeiro_resultado['geometry']['location']

            print(f"Endereço Formatado Oficial: {primeiro_resultado.get('formatted_address')}")
            print(f"Lat: {location['lat']}, Lng: {location['lng']}")
        elif answer.get('status') == 'REQUEST_DENIED':
            print("\n[Erro de Autenticação] A chave de API é inválida ou o faturamento não está ativo no Google Cloud.")
            print(f"Mensagem do Google: {answer.get('error_message')}")
        else:
            print(f"\n[Erro na Busca] O Google retornou o status: {answer.get('status')}")
            print("Verifique se o endereço está correto.")

    # TRATAMENTO DE EXCEÇÕES DE REDE ESPECÍFICAS DO HTTPX
    except httpx.HTTPStatusError as e:
        print(f"\n[Erro de Protocolo] Resposta de erro do servidor (HTTP {e.response.status_code}): {e}")
    except httpx.RequestError as e:
        print(f"\n[Erro de Conexão] Falha de rede ao se comunicar com o Google: {e}")
    except Exception as e:
        print(f"\n[Erro Inesperado] Ocorreu uma falha interna no script: {e}")

# Ponto de entrada obrigatório para rodar códigos assíncronos em Python
if __name__ == "__main__":
    asyncio.run(buscar_coordenadas())
```

Versão 2d: Processamento em lote de múltiplos endereços com HTTPX e Concorrência (**assíncrono**)

- Utilização do método `asyncio.gather(*tarefas)` para disparar e gerenciar múltiplas requisições HTTP de forma paralela e assíncrona, reduzindo drasticamente o tempo total de execução do *script*
- Reaproveitamento de conexões: Instanciação de um único cliente assíncrono comum (`httpx.AsyncClient`) compartilhado entre todas as tarefas, otimizando o reuso de *sockets* TCP e conexões TLS (*keep-alive*)
- Separação lógica clara do código em duas funções principais
 - Uma função utilitária ou *helper*, `geocodificar_endereco`, responsável por tratar individualmente um endereço – como é uma função `async def`, seu nome técnico é **corrotina**
 - Uma função coordenadora (*main*) encarregada de estruturar o lote

Código:

```
import os
import asyncio
import httpx
from dotenv import load_dotenv

load_dotenv()
API_KEY = os.getenv("GOOGLE_API_KEY")
BASE_URL = 'https://maps.googleapis.com/maps/api/geocode/json'

# Função assíncrona responsável por processar UM único endereço utilizando um cliente compartilhado
async def geocodificar_endereco(client: httpx.AsyncClient, endereco: str):
    parameters = {
        'address': endereco,
        'key': API_KEY
    }

    try:
        # Dispara a requisição de forma assíncrona
        response = await client.get(BASE_URL, params=parameters)
        response.raise_for_status()
        answer = response.json()

        status = answer.get('status')

        if status == 'OK' and answer.get('results'):
            primeiro_resultado = answer['results'][0]
            location = primeiro_resultado['geometry']['location']
            print(f"✅ SUCESSO | {endereco} -> Lat: {location['lat']}, Lng: {location['lng']}")
        elif status == 'REQUEST_DENIED':
            print(f"❌ NEGADO | {endereco} -> Chave inválida ou faturamento desativado.")
        else:
            print(f"⚠️ FALHA | {endereco} -> Status da API: {status}")
    except httpx.HTTPStatusError as e:
        print(f"❌ ERRO HTTP | {endereco} -> Código {e.response.status_code}")
    except httpx.RequestError as e:
        print(f"🌐 ERRO REDE | {endereco} -> Falha de conexão: {e}")
    except Exception as e:
        print(f"! ERRO INES | {endereco} -> {e}")

# Função principal que gerencia o lote de tarefas
async def main():
    lista_enderecos = [
        "Rua Paese, 198, Videira, SC, Brazil",
        "Avenida Paulista, 1000, São Paulo, SP, Brazil",
        "Copacabana Palace, Rio de Janeiro, RJ, Brazil",
        "Esplanada dos Ministérios, Brasília, DF, Brazil"
    ]

    print(f"Iniciando a busca simultânea de {len(lista_enderecos)} endereços...\n")

    # Criamos um único cliente assíncrono para ser reaproveitado por todas as requisições (boa prática)
    async with httpx.AsyncClient(timeout=10.0) as client:
        # Cria uma lista de "tarefas" (coroutines) pendentes, uma para cada endereço
        tarefas = [geocodificar_endereco(client, end) for end in lista_enderecos]

        # O asyncio.gather executa todas as tarefas da lista SIMULTANEAMENTE e espera todas terminarem
        await asyncio.gather(*tarefas)

    print("\nTodos os endereços foram processados.")

if __name__ == "__main__":
    asyncio.run(main())
```

Versão 3: Requisição à API de geocodificação (geocoding) da Google com `http.client` (baixo nível)

- Utilização do módulo nativo `http.client.HTTPSConnection` (baixo nível) da biblioteca padrão do Python, realizando a comunicação HTTP sem depender de pacotes de terceiros (como `requests` ou `httpx`)
- Tratamento manual de URL: Uso da função `quote_plus` do módulo `urllib.parse` para codificar caracteres especiais e espaços do endereço, garantindo a formatação correta dos parâmetros exigidos pelo protocolo HTTP
- Ciclo de vida manual da conexão: Abertura explícita do *socket* HTTPS com o host do Google e fechamento obrigatório da conexão por meio do método `connection.close()` após o consumo do fluxo de dados
- Decodificação manual de bytes: Leitura do corpo da resposta em formato bruto binário (bytes) com conversão explícita para texto legível utilizando o padrão de codificação de caracteres UTF-8
- Tratamento especializado de exceções: Tratamento granular de erros divididos entre exceções nativas do protocolo (`HTTPException`) e falhas na desserialização do texto para dicionário Python (`JSONDecodeError`).

Código:

```
import os
import http.client
import json
from urllib.parse import quote_plus
from dotenv import load_dotenv

load_dotenv()
API_KEY = os.getenv("GOOGLE_API_KEY")

endereco = "Rua Paese, 198, Videira, SC, Brazil"

# Codifica o endereço para o formato aceito em URLs (ex: espaços viram '+')
endereco_codificado = quote_plus(endereco)

# Constrói o caminho completo do recurso (Path) injetando a chave protegida
path = f'/maps/api/geocode/json?address={endereco_codificado}&key={API_KEY}'

try:
    print(f"Iniciando conexão HTTPS de baixo nível com o Google...")
    connection = http.client.HTTPSConnection('maps.googleapis.com', timeout=10)

    # Envia o cabeçalho do protocolo GET
    connection.request('GET', path)

    # Captura o fluxo de dados retornado
    response = connection.getresponse()
    raw_data = response.read().decode('utf-8')

    # Fecha a conexão explicitamente após ler os dados recebidos
    connection.close()

    if response.status != 200:
        print(f"Erro no protocolo HTTP: {response.status} {response.reason}")
    else:
        # Transforma o texto bruto de bytes em um dicionário Python
        data = json.loads(raw_data)
        status = data.get('status')

        print(f"\nEndereço pesquisado: {endereco}")
        print(f"Status da API do Google: {status}")

        if status == 'OK':
            if len(data.get('results', [])) > 0:
                location = data['results'][0]['geometry']['location']
                print(f"Endereço Oficial Google: {data['results'][0].get('formatted_address')}")
                print(f"Latitude: {location['lat']}")
                print(f"Longitude: {location['lng']}")
            else:
                print("Erro: Nenhum resultado geográfico retornado (lista vazia).")
        elif status == 'REQUEST_DENIED':
            print("\n[Acesso Negado] Verifique sua chave no arquivo .env ou as restrições de IP no painel do Google.")
            print(f"Detalhe do erro: {data.get('error_message', 'Sem mensagem detalhada.')}")
        else:
            print(f"Falha na busca do Google: {status}")
except http.client.HTTPException as e:
    print(f"Erro de comunicação na camada HTTP: {e}")
except json.JSONDecodeError as e:
    print(f"Erro crítico: O servidor não retornou um formato JSON válido: {e}")
except Exception as e:
    print(f"Erro inesperado no script: {e}")
```

Versão 4a: Comunicação com a API de geocodificação da Google via *sockets* criptografados (camada de transporte)

- Comunicação em nível de transporte: Utilização do módulo nativo *socket* para estabelecer uma conexão direta via protocolo TCP (*SOCK_STREAM*), operando abaixo de qualquer biblioteca HTTP clássica do Python
- Criptografia manual da sessão (SSL/TLS): Uso do módulo *ssl* para criar um contexto de segurança padrão e envelopar o *socket* bruto (*context.wrap_socket*), realizando o aperto de mão (*handshake*) criptográfico obrigatório para a porta 443
- Construção manual do protocolo HTTP: Montagem puramente textual e manual dos cabeçalhos da requisição baseada na especificação estrita do protocolo HTTP/1.1, incluindo a terminação obrigatória de linhas com `\r\n`
- Otimização de encerramento: Injeção do cabeçalho `Connection: close` no corpo do texto para instruir o servidor a encerrar voluntariamente o canal de comunicação assim que o último byte do JSON for transmitido
- Leitura de rede por blocos (*buffer*): Implementação de um laço de repetição (`while True`) com o método `recv(4096)` para consumir e decodificar em tempo real o fluxo contínuo de pacotes binários vindos da rede
- Configuração de limite de tempo físico (`settimeout(10.0)`) diretamente no *socket* TCP para blindar o *script* contra *loops* infinitos causados por quedas ou instabilidades na infraestrutura de rede

Código:

```
import os
import socket
import ssl
from urllib.parse import quote_plus
from dotenv import load_dotenv

load_dotenv()
API_KEY = os.getenv("GOOGLE_API_KEY")

HOST = 'maps.googleapis.com'
PORT = 443
endereco = "Rua Paese, 198, Videira, SC, Brazil"

# Montagem textual da requisição HTTP/1.1 estrita (Sem o parâmetro obsoleto 'sensor')
request_text = (
    f"GET /maps/api/geocode/json?address={quote_plus(endereco)}&key={API_KEY} HTTP/1.1\r\n"
    f"Host: {HOST}\r\n"
    f"User-Agent: geo_raw_socket_v6\r\n"
    f"Connection: close\r\n" # Força o Google a fechar o socket após o fim do envio
    f"\r\n"
)

try:
    print(f"Iniciando conexão segura via Socket com {HOST}...")

    # Configuração manual da camada de Transporte e Criptografia
    context = ssl.create_default_context()
    raw_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    raw_socket.settimeout(10.0) # Evita travamento infinito do script

    secure_socket = context.wrap_socket(raw_socket, server_hostname=HOST)
    secure_socket.connect((HOST, PORT))

    print("Enviando requisição HTTP...")
    secure_socket.sendall(request_text.encode('utf-8'))

    print("Recebendo bytes do servidor e exibindo em tempo real:\n")
    print("=" * 50 + " INÍCIO DOS DADOS BRUTOS " + "=" * 50)

    # Laço de repetição que lê os pacotes da rede à medida que eles chegam
    while True:
        # Lê blocos de até 4096 bytes por vez
        dados_recebidos = secure_socket.recv(4096)

        # Se o servidor fechar a conexão (fim dos dados), o retorno será vazio
        if not dados_recebidos:
            break

        # Decodifica e imprime o pedaço do fluxo de rede imediatamente na tela
        print(dados_recebidos.decode('utf-8'), end='')

    print("\n" + "=" * 51 + " FIM DOS DADOS BRUTOS " + "=" * 51)

    secure_socket.close()
except socket.timeout:
    print("\n[Erro de Rede] Tempo limite esgotado (Timeout) ao aguardar resposta do servidor.")
except Exception as e:
    print(f"\n[Erro Inesperado] Falha na comunicação via Socket: {e}")
```

Versão 4b: Diagnóstico e validação prática de **Chunked Transfer-Encoding** via servidor de teste **HTTPBin**

- Redirecionamento do fluxo de testes para o serviço `httpbin.org/stream/3`, um endpoint projetado especificamente para forçar a divisão e entrega da resposta HTTP em múltiplos fragmentos (*chunks*).
- Uso de *sockets* em nível de transporte para capturar a estrutura exata e os metadados textuais enviados pela placa de rede, servindo como ambiente experimental de validação para o algoritmo de desmembramento.
- Mecanismo de interrupção por marcador: Implementação de uma verificação lógica baseada em bytes brutos (`b'\r\n0\r\n\r\n'`) dentro do laço de leitura, permitindo identificar o sinalizador universal de encerramento de transmissão *chunked* sem depender do fechamento do socket pelo servidor.
- Impressão incremental em fluxo: Leitura contínua por meio de buffers fixos de 4096 bytes com decodificação direta para texto (utf-8), permitindo visualizar a chegada dos dados e a alternância dos tamanhos dos pacotes em tempo real na tela.

Código:

```
import os
import socket
import ssl
from urllib.parse import quote_plus
from dotenv import load_dotenv

load_dotenv()
API_KEY = os.getenv("GOOGLE_API_KEY")

HOST = 'httpbin.org'
PORT = 443

# Este endpoint do httpbin gera uma resposta dividida obrigatoriamente em 3 pedaços (chunks)
request_text = (
    "GET /stream/3 HTTP/1.1\r\n"
    f"Host: {HOST}\r\n"
    "User-Agent: teste_acadêmico_v8\r\n"
    "Connection: close\r\n" # Pode usar close aqui que ele vai quebrar do mesmo jeito!
    "\r\n"
)

try:
    context = ssl.create_default_context()
    raw_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    raw_socket.settimeout(5.0) # Importante: Como a conexão fica viva, precisamos do timeout para sair do loop

    secure_socket = context.wrap_socket(raw_socket, server_hostname=HOST)
    secure_socket.connect((HOST, PORT))
    secure_socket.sendall(request_text.encode('utf-8'))

    print("=" * 40 + " DADOS BRUTOS (FORCED CHUNK) " + "=" * 40)

    while True:
        try:
            dados_recebidos = secure_socket.recv(4096)
            if not dados_recebidos:
                break

            # Imprime os bytes brutos exatamente como chegam da placa de rede
            print(dados_recebidos.decode('utf-8'), end='')

            # Se encontrar o chunk final (um 0 isolado seguido de quebras de linha), encerra o laço
            if b'\r\n0\r\n\r\n' in dados_recebidos:
                print("\n\n[Socket] Detectado o marcador de fim de chunk (0). Encerrando leitura.")
                break
        except socket.timeout:
            # Como usamos 'keep-alive', o Google não fecha a conexão sozinho imediatamente,
            # o timeout garante que o script não fique travado após receber tudo.
            print("\n\n[Socket] Timeout atingido. Conexão mantida viva pelo servidor.")
            break

    secure_socket.close()
    print("=" * 109)
except Exception as e:
    print(f"\nErro: {e}")
```

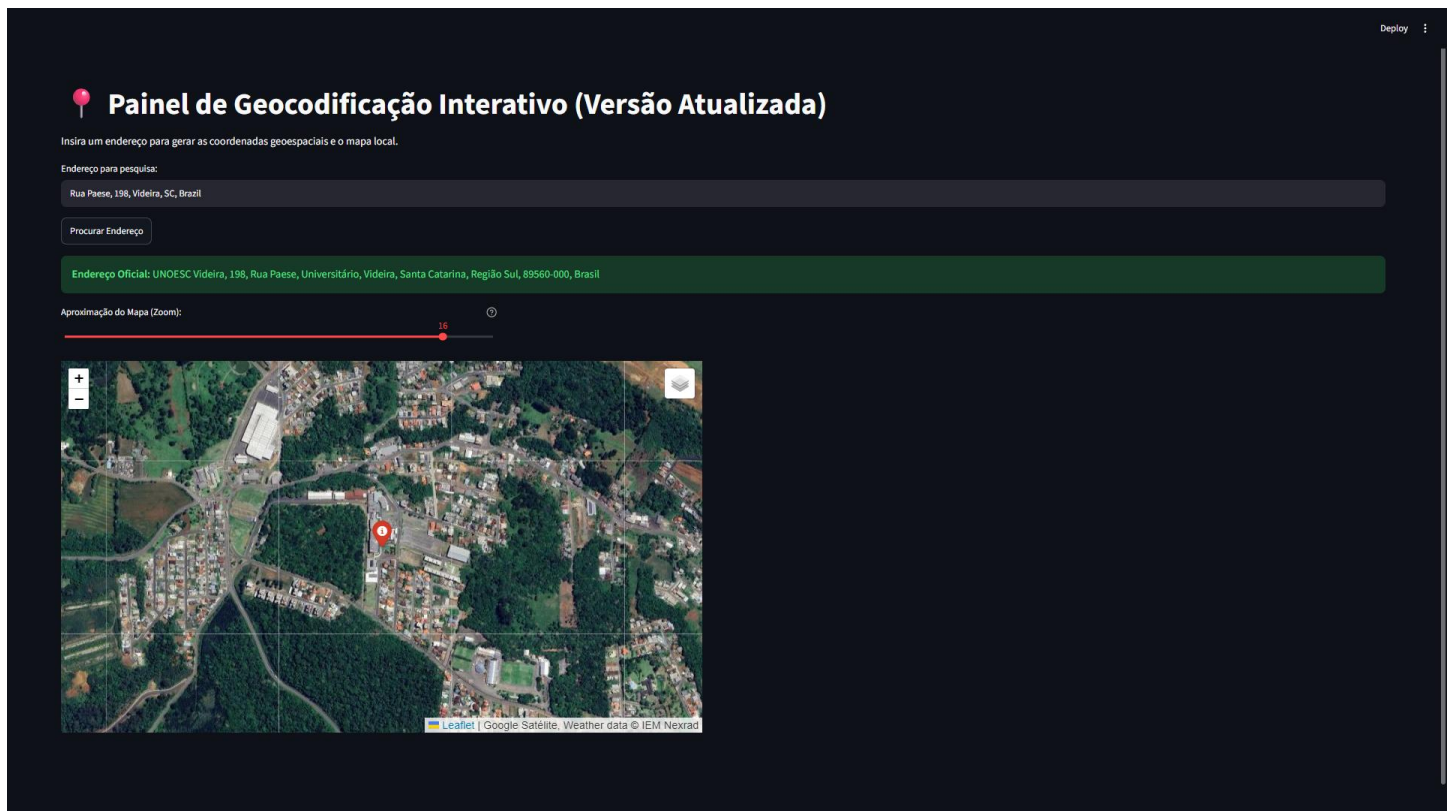
Versão 5a: *Interface gráfica web* com Streamlit, geocodificação gratuita (Geopy) e mapa interativo (Folium)

- Construção de *dashboard* ágil: Utilização do *framework* `streamlit` para conceber uma *interface web* completa diretamente em Python, eliminando a necessidade de codificar componentes em HTML, CSS ou JavaScript
- Geocodificação *open-source* sem custo: Integração com a biblioteca `geopy` utilizando o provedor `Nominatim` (baseado no ecossistema OpenStreetMap), permitindo realizar buscas de coordenadas geográficas de forma gratuita e sem a obrigatoriedade de chaves de API pagas
- Persistência de estado (*session state*): Emprego da estrutura nativa `st.session_state` para reter em memória os dados do endereço encontrado, garantindo que o mapa permaneça renderizado na tela mesmo após o clique de botões ou novas interações do usuário.
- Mapeamento multicamadas (*multi-tile layers*): Utilização do `folium` para renderizar mapas dinâmicos e sobrepor diferentes camadas cartográficas selecionáveis pelo usuário (Google Satélite, Google Maps de Ruas e um radar meteorológico em tempo real alimentado pela API do `RainViewer`)
- Renderização síncrona: Aplicação da função `folium_static` da biblioteca auxiliar `streamlit_folium` para forçar a renderização do mapa como um bloco HTML fixo, otimizando o carregamento da página e mitigando problemas de reinicialização do `iframe`
- Inclusão preventiva de uma diretiva de infraestrutura (`WindowsSelectorEventLoopPolicy`) para reduzir falhas físicas de quebra de conexão (`ConnectionResetError [WinError 10054]`) ao rodar o *loop* assíncrono do interpretador no sistema operacional Windows
- Tratamento de exceções de terceiros: Abordagem granular no tratamento de falhas focada em instabilidades e indisponibilidades do servidor remoto do OpenStreetMap com capturas específicas `GeocoderUnavailable` e `GeocoderServiceError`.

Para executar:

➤ `streamlit run nome_programa.py`

Resultado:



Código:

```
import streamlit as st
import folium
import sys
import asyncio
from geopy.geocoders import Nominatim
from geopy.exc import GeocoderUnavailable, GeocoderServiceError
from streamlit_folium import folium_static

# Corrige o erro ConnectionResetError [WinError 10054] no Windows
if sys.platform == 'win32':
    asyncio.set_event_loop_policy(asyncio.WindowsSelectorEventLoopPolicy())

def geocode(endereco):
    locator = Nominatim(user_agent="app_geo_streamlit_static_v3 (seu_email@provedor.com)")
    try:
        local = locator.geocode(endereco)
        return local
    except (GeocoderUnavailable, GeocoderServiceError) as e:
        st.error(f"Erro no serviço de geocodificação: {e}")
        return None

st.set_page_config(page_title="Geocodificação Estática Python", layout="wide")
st.title("♀ Aplicação de Geocodificação Estática com Radar")
st.markdown("Digite um endereço para renderizar a visualização em mapa com radar de chuva.")

endereco = st.text_input(
    "Endereço:", value="Rua Paese, 198, Videira, SC, 89566-252, Brazil", help="Incluir o país (Brazil) melhora a precisão."
)

# Inicializa o estado da sessão para persistir os dados e o mapa não sumir ao interagir
if "local_data" not in st.session_state:
    st.session_state.local_data = None

if st.button("Procurar"):
    if not endereco:
        st.warning("Por favor, digite um endereço.")
    else:
        with st.spinner('Buscando coordenadas...'):
            local = geocode(endereco)
            if local:
                st.session_state.local_data = {"lat": local.latitude, "lon": local.longitude, "address": local.address}
            else:
                st.session_state.local_data = None
                st.error("✖ Endereço não encontrado.")

# --- RENDERIZAÇÃO ESTÁTICA DO MAPA (FORA DO BOTÃO) ---
if st.session_state.local_data:
    data = st.session_state.local_data
    st.success(f"***Endereço encontrado:** {data['address']}")

    # Inicializa o mapa estruturado
    m = folium.Map(location=[data['lat'], data['lon']], zoom_start=11, tiles=None)

    # Camada 1: Google Satélite
    folium.TileLayer(
        tiles='https://{s}/{x}/{y}/{z}', attr='Google Satélite',
        name='Satélite', max_zoom=20, subdomains=['mt0', 'mt1', 'mt2', 'mt3']
    ).add_to(m)

    # Camada 2: Google Maps Padrão
    folium.TileLayer(
        tiles='https://{s}/{x}/{y}/{z}', attr='Google Maps',
        name='Mapa de Ruas', max_zoom=20, subdomains=['mt0', 'mt1', 'mt2', 'mt3']
    ).add_to(m)

    # Camada 3: Radar de chuva global e gratuito da RainViewer
    folium.TileLayer(
        tiles='https://rainviewer.com{z}/{x}/{y}/2/1_1.png',
        attr='Radar de Chuva © RainViewer',
        name='Radar de Chuva (Tempo Real)',
        overlay=True, control=True, opacity=0.6
    ).add_to(m)

    # Adiciona o menu controlador de camadas no mapa
    folium.LayerControl().add_to(m)

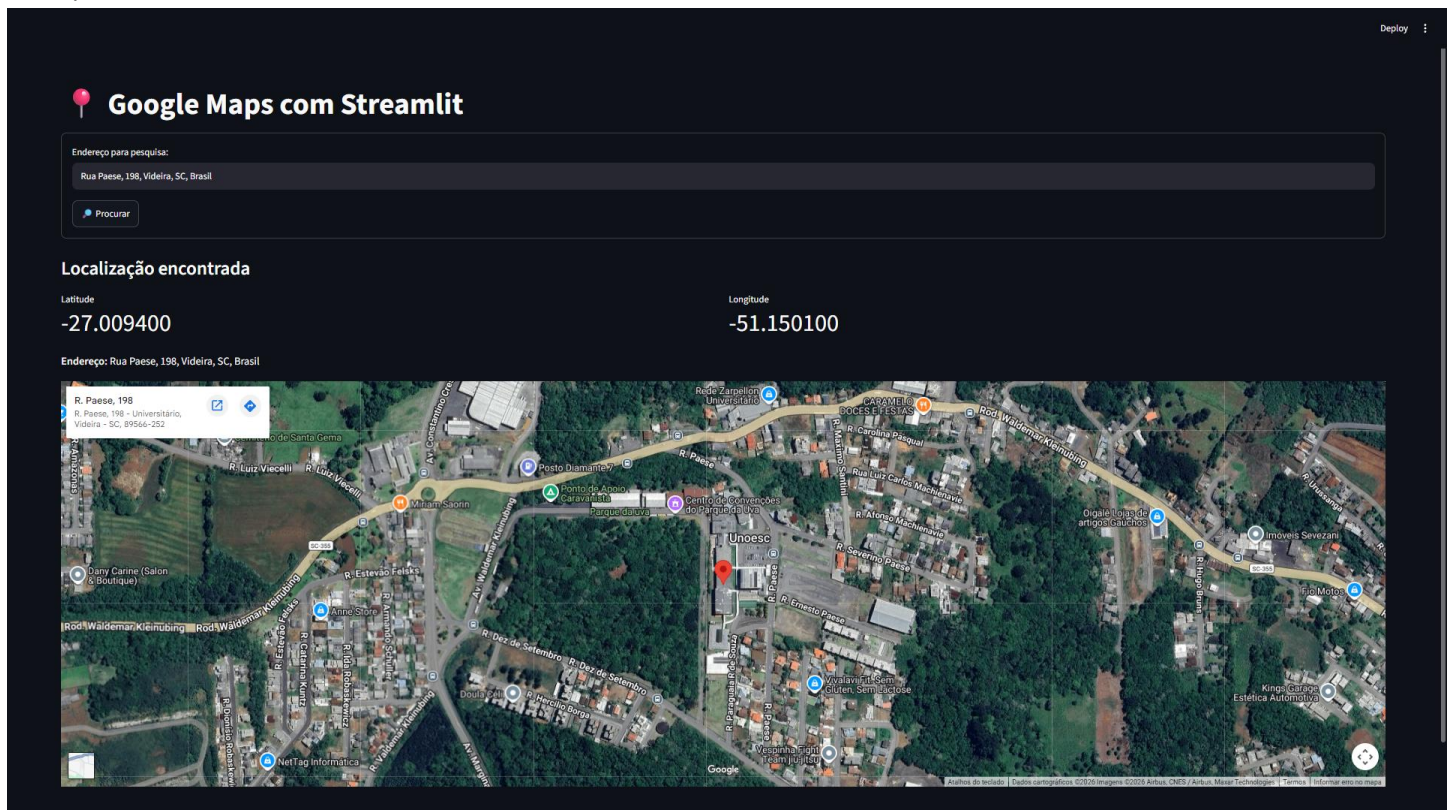
    # Adiciona o marcador do local encontrado
    folium.Marker(
        location=[data['lat'], data['lon']], popup=f"<b>{data['address']}</b>",
        tooltip="Clique para ver o endereço", icon=folium.Icon(color='red', icon='info-sign')
    ).add_to(m)

    # RENDIMENTO ESTÁTICO: Converte o objeto do mapa em representação HTML rígida para a página
    folium_static(m, width=800, height=600)
```

Versão 5b: Interface gráfica com Streamlit, Geocodificação (`urllib`) e incorporação oficial (Google Maps *Embed* API)

- Utilização dos módulos de biblioteca padrão `urllib.request` e `urlopen` para efetuar chamadas HTTP síncronas de alto nível, eliminando totalmente a dependência de pacotes externos como `requests` ou `httpx`
- Mecanismo de otimização e *cache*: Emprego do decorador `@st.cache_data(show_spinner=False)` para memorizar e reaproveitar os resultados de buscas idênticas, reduzindo custos financeiros de chamadas repetitivas à API do Google Cloud
- Componentização via formulário: Agrupamento do campo de entrada e do botão através de `st.form`, reduzindo a reexecução do Streamlit a cada caractere digitado e disparando o fluxo apenas no clique do botão de submissão
- Persistência atômica de estado: Uso da estrutura `st.session_state` fatiada em chaves isoladas (`latitude`, `longitude`, `endereço_atual`, `place_id`) para fixar os dados da última busca bem-sucedida da aplicação
- Validação granular da API de negócio: Tratamento lógico explícito para os *status* de resposta da API do Google, isolando cenários de buscas vazias (`ZERO_RESULTS`) e extraindo parâmetros estritos de localização (`place_id`)
- Monitoramento e exceções de rede: Captura especializada dividida entre erros de protocolo de aplicação (`HTTPError`), falhas de conexão de infraestrutura física (`URLError`) e interrupções por limite de tempo (`socket.timeout`)
- Segurança e interrupção preventiva: Validação da existência da variável de ambiente no arquivo `.env` com interrupção forçada (`st.stop()`), impedindo chamadas malformadas e vazamento de *tokens*
- Exibição gráfica atualizada (2026): Substituição de bibliotecas de mapeamento de terceiros pela renderização direta e oficial do `Iframe` do Google Maps via `st.iframe`, garantindo total compatibilidade com os padrões vigentes do navegador
- Apresentação de indicadores dinâmicos: Exibição dos dados geodésicos formatados por meio do componente nativo `st.metric`, separando visualmente os eixos cartesianos.

Exemplo:



Código:

```
import os
import socket
import ssl
import json

from urllib.parse import quote_plus
from urllib.request import urlopen, Request
from urllib.error import URLError, HTTPError

import streamlit as st
from dotenv import load_dotenv

load_dotenv()
API_KEY = os.getenv("GOOGLE_API_KEY")

st.set_page_config(
    page_title="Google Maps com Streamlit",
    page_icon="📍",
    layout="wide"
)

st.title("📍 Google Maps com Streamlit")

if not API_KEY:
    st.error("A variável GOOGLE_API_KEY não foi encontrada no arquivo .env.")
    st.stop()

@st.cache_data(show_spinner=False)
def geocode_google(endereco):
    endereco_codificado = quote_plus(endereco)
    url = (
        "https://maps.googleapis.com/maps/api/geocode/json"
        f"?address={endereco_codificado}"
        f"&key={API_KEY}"
        "&language=pt-BR"
        "&region=br"
    )

    contexto_ssl = ssl.create_default_context()

    requisicao = Request(
        url,
        headers={
            "User-Agent": "StreamlitGoogleMaps/1.0",
            "Accept": "application/json"
        }
    )

    try:
        with urlopen(
            requisicao,
            timeout=10,
            context=contexto_ssl
        ) as resposta:
            corpo = resposta.read().decode("utf-8")
    except HTTPError as erro:
        raise Exception(f"Erro HTTP ao acessar o Google: {erro.code}")
    except URLError as erro:
        raise Exception(f"Erro de conexão com o Google: {erro.reason}")
    except socket.timeout:
        raise Exception("Tempo limite excedido ao acessar o Google.")

    dados = json.loads(corpo)
    status = dados.get("status")

    if status == "ZERO_RESULTS":
        return None

    if status != "OK":
        mensagem = dados.get("error_message", "Erro não informado pelo Google.")
        raise Exception(f"Google Geocoding API: {status} - {mensagem}")

    resultado = dados["results"][0]
    coordenadas = resultado["geometry"]["location"]

    return {
        "latitude": coordenadas["lat"],
```

```

        "longitude": coordenadas["lng"],
        "endereco": resultado["formatted_address"],
        "place_id": resultado.get("place_id")
    }

if "endereco_atual" not in st.session_state:
    st.session_state.endereco_atual = ("Rua Paese, 198, Videira, SC, Brasil")

if "latitude" not in st.session_state:
    st.session_state.latitude = -27.0094

if "longitude" not in st.session_state:
    st.session_state.longitude = -51.1501

if "place_id" not in st.session_state:
    st.session_state.place_id = None

with st.form("formulario_pesquisa"):
    endereco = st.text_input(
        "Endereço para pesquisa:",
        value=st.session_state.endereco_atual
    )

    procurar = st.form_submit_button("🔍 Procurar")

if procurar:
    if not endereco.strip():
        st.warning("Digite um endereço para realizar a pesquisa.")
    else:
        try:
            with st.spinner("Consultando Google Geocoding API..."):
                local = geocode_google(endereco.strip())

                if local:
                    st.session_state.latitude = (local["latitude"])
                    st.session_state.longitude = (local["longitude"])
                    st.session_state.endereco_atual = (local["endereco"])
                    st.session_state.place_id = (local["place_id"])
                else:
                    st.warning("Endereço não encontrado.")
        except Exception as erro:
            st.error(str(erro))

st.subheader("Localização encontrada")
col1, col2 = st.columns(2)

with col1:
    st.metric(
        "Latitude",
        f"{st.session_state.latitude:.6f}"
    )

with col2:
    st.metric(
        "Longitude",
        f"{st.session_state.longitude:.6f}"
    )

st.write(
    "**Endereço:**",
    st.session_state.endereco_atual
)

if st.session_state.place_id:
    consulta_mapa = quote_plus("place_id:" + st.session_state.place_id)
else:
    consulta_mapa = quote_plus(st.session_state.endereco_atual)

url_mapa = (
    "https://www.google.com/maps/embed/v1/place"
    f"?key={API_KEY}"
    f"&q={consulta_mapa}"
    "&language=pt-BR"
    "&region=BR"
)

st.iframe(url_mapa, height=600)

```